EPFL HACKATHON 2026 · BELEM CAPITAL CHALLENGE

DeFi Risk from Scratch

A complete course on every blockchain and DeFi concept behind the DeFi Risk Dependency project.

No prior knowledge assumed.



Based on the repository at commit ff651d1 (origin/main). Real numbers come from the repository's cached data snapshots.

Contents

Part 1. Blockchains from scratch **4**
1. Why blockchains exist 5
2. How a blockchain works 6
3. Wallets, keys and addresses 7
4. Transactions, gas, nodes and explorers 8
5. The chains in this project 9
6. Smart contracts 10
7. Who controls a contract: proxies, owners, timelocks 11
8. Tokens 12
9. Bridges: moving tokens between chains 14

Part 2. DeFi building blocks **16**
10. DeFi, protocols and DeFiLlama 17
11. Exchanges and AMMs: where tokens are traded 18
12. Oracles: how contracts learn prices 19
13. Lending markets: the heart of the project 20
14. Vaults and curators 22
15. Pendle: splitting yield from principal 23
16. Ethena and USDe: a dollar built from hedges 24
17. Perpetual futures and Hyperliquid 25
18. Putting it together: the dependency map of a position 26

Part 3. Risk: from a map to numbers **27**
19. Incidents: what can go wrong 28
20. A probability toolkit 28
21. Calibration: where the numbers come from 30
22. From the graph to your loss: exposure and the stress engine 31
23. Liquidity risk: can you get out, and at what price? 34

Part 4. Decisions: STAY or EXIT **36**
24. The STAY / EXIT rule 37
25. When to leave a loan: the exit trigger 38
26. The whole book: what pockets share 40

Part 5. The project's code **42**
27. The pipeline, file by file 43
28. Running and reading the app 45
29. Honest limits and where to go next 46

Part A. Appendices **47**
A. Formula sheet 48
B. Glossary 48
C. Answers to Check yourself 52

How to use this course

This course is for anyone who wants to understand the **DeFi Risk Dependency** project, and the world it works in, **starting from zero**. You do not need to know what a blockchain is. Every technical term is defined the first time it appears and collected in the glossary (Appendix B).

What the project does, in one paragraph. A fund (Belem Capital) holds several DeFi positions, called **pockets**. For each one, the project automatically discovers what the position depends on (the blockchain, the protocols, the token issuers, the price oracles, the markets, the lenders). It then estimates how much money would be lost if any of them failed, and answers the risk manager's question: **STAY or EXIT**, and why. It runs on real, public data.

The path

Part	You will learn	Chapters
1. Blockchains from scratch	Ledgers, blocks, keys, transactions, chains, smart contracts, admins, tokens, bridges.	1-9
2. DeFi building blocks	Exchanges and AMMs, oracles, lending, vaults, Pendle, Ethena, perpetual futures; the dependency map.	10-18
3. Risk	Incidents, probability, calibration on history, how losses spread through the graph, liquidity.	19-23
4. Decisions	The STAY / EXIT rule, the exit trigger for loans, the whole book.	24-26
5. The code	Where each concept lives in the code, how to run and read the app, limits.	27-29

The boxes

Coloured boxes appear throughout. Each colour has one job:

KEY IDEA	The one thing to remember from a section.
EVERYDAY ANALOGY	A comparison with everyday life to build intuition.
WORKED EXAMPLE	A calculation done step by step. Redo it yourself.
REAL NUMBERS FROM THIS PROJECT	Figures produced by the project's own code on real data.
IN THE PROJECT'S CODE	Where the concept lives in the code.
WATCH OUT	A trap, a subtlety, or a limit of the model.
CHECK YOURSELF	Questions to test yourself. Answers in Appendix C.

The running examples

- **PT-sUSDe 26NOV2026**: a Pendle principal token on Ethereum, the hackathon's reference position (address `0x47ad…ef77e`).
- **Huma USDC Main**: a Morpho vault lending USDC against PST, a real-world-asset token bridged from Solana. It shows almost every danger at once.
- **The demo book**: six pockets of 1M USD each (PT-sUSDe, RockawayX, Huma, Belem RWA EURCV, Kamino Steakhouse HY USDC, Kamino RWA USDC). Figures come from the repository's cached snapshots; live runs will differ slightly, the reasoning will not.

PART 1

Blockchains from scratch

What a blockchain is, why it exists, and the pieces this project reads every day: chains, wallets, smart contracts, tokens and bridges.

1. Why blockchains exist

Before any technology, start with a simple observation: **money is a record**. When your bank says you have 1,000 euros, there is no pile of notes with your name on it. There is a line in the bank's books (its **ledger**) that says *this person is owed 1,000 euros*. When you pay someone, the bank edits two lines: yours goes down, theirs goes up.

This works because everyone **trusts the bank** to keep the ledger honestly, not to lose it, and not to edit it in its own favour. Most of the time that trust is deserved. But it has costs: the bank decides who may open an account, can freeze it, works office hours across borders, and if the bank fails, its ledger fails with it.

The problem a blockchain solves

Could we keep a ledger that **nobody owns** and that **nobody can cheat**? Digital information is easy to copy. If money were just a file, you could send the same file to two people. This is called the **double-spending problem**. A bank solves it by being the single referee. A blockchain solves it without a referee.

> **EVERYDAY ANALOGY**
>
> Imagine a village where every family keeps an identical notebook listing who owns what. When Alice pays Bob, she announces it in the square. Every family writes it down. If Alice later claims she never paid, the whole village can show her the line. If one family tries to secretly edit its notebook, it no longer matches the thousands of other copies, and everyone ignores it.
>
> A blockchain is that notebook, copied on thousands of computers around the world, with strict rules for adding new lines.

Bitcoin (2009) was the first working blockchain. Its ledger only tracks one thing: who owns which bitcoins. **Ethereum** (2015) went further: its ledger can also store **programs** that hold money and follow rules automatically. These programs are called **smart contracts** (chapter 6). Everything in this project, and all of "DeFi", is built on that idea.

> **KEY IDEA**
>
> A blockchain is a **shared ledger** that nobody owns, that anyone can read, and that cannot be edited after the fact. It replaces *trust in a referee* with *rules enforced by thousands of computers*.

> **CHECK YOURSELF**
>
> - Why is "money is a record" the right starting point to understand blockchains?
> - What is double spending, and how does a bank prevent it?

2. How a blockchain works

Transactions and blocks

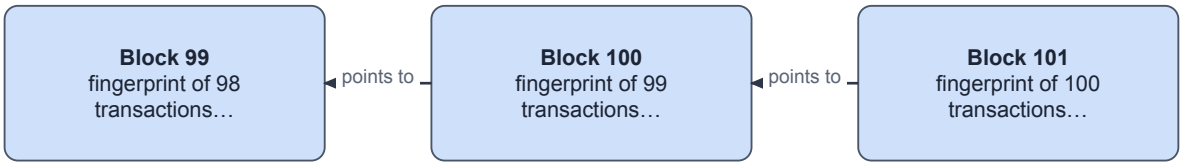
A **transaction** is one instruction for the ledger, for example *send 100 USDC from address A to address B*. Transactions are not added one by one. They are grouped into **blocks**, like pages of the notebook. On Ethereum a new block appears roughly every 12 seconds. On Solana it is a fraction of a second.

Fingerprints that chain the blocks

Each block carries a **hash** of the block before it. A hash is a *digital fingerprint*: a short code (for example `0x9f2c…e41a`) computed from any amount of data. Two key properties:

- The same data always gives the same fingerprint.
- Changing even one comma in the data gives a completely different fingerprint.

Because block 101 contains the fingerprint of block 100, which contains the fingerprint of block 99, and so on, editing an old block would change its fingerprint, which would break every block after it. That is the "chain" in blockchain: a tamper-evident sequence of pages.



Each block stores the fingerprint (hash) of the previous one. Change an old block and every later fingerprint stops matching.

Who adds the next block? Validators and consensus

Someone must decide which transactions go into the next block. On modern chains like Ethereum this job belongs to **validators**: computers run by people who have locked up ("staked") their own money as a guarantee of good behaviour. If a validator cheats, part of its stake is destroyed ("slashed"). This system is called **proof of stake**. Bitcoin uses an older system, **proof of work**, where computers compete to solve puzzles.

The rules by which validators agree on the next block are called **consensus**. Once enough validators have agreed on a block, it is considered **final**: it will not be undone.

When things go wrong: a chain halt

If validators stop agreeing (a software bug, a crash, a network attack), no new blocks are produced. The chain **halts**. Nobody can move money until it restarts. Solana halted several times in 2022, for hours each time. A halt rarely destroys money, but you are frozen while prices keep moving elsewhere.

> **IN THE PROJECT'S CODE**
>
> Every position in the project has an edge to a **chain** node ("settles_on"). In `src/risk/assumptions.py`, a chain halt is a **JUDGEMENT** scenario (an expert estimate, because no public dataset of halts exists): **1% chance per year**, costing **2%** of the value (the price move while you are stuck).

CHECK YOURSELF

- What would happen to block 101 if someone edited a transaction inside block 99?
- Why does a validator have money at stake?
- Why is a chain halt usually a small loss rather than a total loss?

3. Wallets, keys and addresses

On a blockchain there are no names or bank accounts. There are **addresses**, and whoever holds the matching **private key** controls what is at that address.

Private key, address, signature

- **Private key**: a huge secret number, created randomly. Whoever knows it controls the money. It is never shared. It is often written down as a **seed phrase** of 12 or 24 ordinary words.
- **Address**: a public identifier computed from the key, like an account number. On Ethereum-style chains it looks like `0x47ad2cd1dd15739a7a035b9d3b7828d916fef77e`: "0x" followed by 40 characters. On Solana it looks different, for example `DWSXb18xZApz29vnQpgR2m6MynCT7PznaXt7Ut7M7KaP`.
- **Signature**: proof, attached to a transaction, that it was approved by the owner of the private key. Anyone can check a signature. Nobody can forge one without the key.

EVERYDAY ANALOGY

An address is like a glass mailbox in a public square: everyone can see what is inside and anyone can drop money in. Only the owner of the key can take money out. Lose the key and the money is stuck forever. Let someone steal it and they can empty the box, with no bank to call.

Hexadecimal, in one minute

Addresses and blockchain data are written in **hexadecimal** ("hex"): a way of writing numbers with 16 digits, 0-9 then a-f. The prefix `0x` just means "what follows is hex". So `0x10` is 16, `0xff` is 255. You do not need to compute in hex. You just need to recognise it.

Who can hold a key: wallets, multisigs and contracts

Not every address belongs to a person. The project distinguishes:

Kind of controller	What it is	Risk intuition
EOA (externally owned account)	An ordinary wallet: one private key.	One stolen or lost key = full control lost. The project flags it.
Multisig (e.g. a "Safe 4/8")	A smart contract that needs *k* of *n* keyholders to approve. "4/8" means 4 of 8 people must sign.	Much safer: an attacker must steal several keys. A low threshold (1 or 2) is flagged.
Timelock	A contract that delays every action, e.g. by 2 days, before it takes effect.	Gives users time to see a change coming and withdraw.
Contract	Another program that holds the role.	Look at who controls *that* program.

IN THE PROJECT'S CODE

`src/graph/crawler.py`, function `controller()`, asks an address a few questions on-chain. Does it have code? If not, it is a plain wallet (EOA). Does it answer `getThreshold()` and `getOwners()`? Then it is a Safe multisig. Does it answer `getMinDelay()`? Then it is a timelock. `control_flags()` then writes the red flags you see in the app, for example *"owner is a single wallet: one stolen key controls it"*.

CHECK YOURSELF

- What is the difference between a private key and an address?
- Why is a 4-of-8 multisig safer than a single wallet as the owner of a vault?

4. Transactions, gas, nodes and explorers

Writing costs gas, reading is free

Changing the ledger (sending tokens, depositing in a vault) requires a **transaction**: a signed instruction broadcast to the network. The sender pays a fee called **gas**, which pays validators for the computing work. Gas is paid in the chain's own coin (ETH on Ethereum, SOL on Solana).

Reading the ledger is free. Anyone can ask *what is the balance of this address?* or *what price does this oracle report?* without paying, because nothing changes. This project only ever **reads**. It never sends a transaction or moves any money.

Nodes and RPC: how a program asks the blockchain a question

A **node** is a computer that keeps a full copy of the ledger. To ask a question, a program sends a message to a node over the internet in a standard format called **JSON-RPC** (RPC = *remote procedure call*, "run this function on your computer and send me the answer"). Public providers run free nodes. The project uses `ethereum-rpc.publicnode.com` and similar.

RPC method	Plain meaning	Used by the project for
`eth_call`	Run a read-only function of a contract and return the answer.	Read an oracle's price, a vault's `asset()`, an owner's address…
`eth_getCode`	Return the program code stored at an address (empty for a wallet).	Tell a wallet (EOA) from a contract; detect minimal proxies.
`eth_getStorageAt`	Read one raw memory slot of a contract.	Find who can upgrade a contract (proxy admin slot, chapter 7).

IN THE PROJECT'S CODE

`src/data/chain.py` holds the RPC addresses per chain and a `batch()` helper that sends many questions in **one** request (faster and kinder to free nodes). Every answer goes through `src/data/cache.py`, which stores it in `data/cache/`. If the node is down during a demo, the stored answer is used. `DEFI_OFFLINE=1` forces this.

Block explorers: the public window

A **block explorer** is a website that shows the ledger in a readable form: every transaction, every balance, every contract. **Etherscan** for Ethereum, **Arbiscan** for Arbitrum, **Basescan**, **BscScan**, **Solscan** for Solana… The project uses explorer links as **evidence** on graph edges. Anyone can click and check that a dependency is real.

> **CHECK YOURSELF**
> - Why does this project never pay gas?
> - What does `eth_getCode` return for an ordinary wallet, and why is that useful?

5. The chains in this project

There is not one blockchain but many. Each is a separate ledger with its own validators, fees and speed. Money on one chain does not automatically exist on another (moving it requires a **bridge**, chapter 9).

Layer 1, Layer 2, and the EVM

- **Layer 1 (L1)**: an independent blockchain with its own validators (Ethereum, Solana, BSC, Hyperliquid L1).
- **Layer 2 (L2)**: a faster, cheaper chain that runs on top of an L1 and regularly posts summaries of its activity to it, inheriting much of its security. Arbitrum and Base are L2s of Ethereum ("rollups").
- **EVM** (Ethereum Virtual Machine): the "computer" that runs smart contracts on Ethereum. Many chains copy it, so the same contracts, addresses format and tools work there too. These are **EVM chains**. Solana is not an EVM chain: different programs, different address format, different tools.
- **Chain ID**: a number that identifies an EVM chain (1 = Ethereum). The project uses it everywhere to know which node to ask.

Chain	Chain ID	Type	Why it is in the project
Ethereum	1	L1, EVM	The main DeFi chain: Pendle, Morpho, Aave, Ethena.
Arbitrum	42161	L2 of Ethereum, EVM	Pendle and Aave markets. Also holds the USDC deposited on Hyperliquid (its bridge).
Base	8453	L2 of Ethereum (run by Coinbase), EVM	Morpho and Aave markets.
BSC (BNB Smart Chain)	56	L1 linked to Binance, EVM	Pendle and Aave markets.
HyperEVM / Hyperliquid L1	999	L1, EVM side + trading engine	Hyperliquid perp vaults (chapter 17).
Plasma	9745	L1 for stablecoin payments, EVM	Pendle and Aave markets.
Solana	101 (project's own label)	L1, **not** EVM	Kamino vaults and reserves. Solana has no EVM chain ID, so the project uses 101.

> **WATCH OUT**
> The same token name on two chains can be two very different things. "USDC on Arbitrum" is issued natively by Circle. A token copied to another chain by a bridge is only as good as that bridge. The project tracks this with `bridged_by` and `backed_on` edges.

> **CHECK YOURSELF**
> - What is the difference between Ethereum and Arbitrum?
> - Why can't the project read Solana with the same RPC helpers it uses for Base?

6. Smart contracts

A **smart contract** is a program stored at an address on the blockchain. It can hold money and it follows its code exactly, automatically, for anyone who calls it. Nobody can stop it or make an exception, unless the code itself allows it.

> **EVERYDAY ANALOGY**
> A vending machine: insert 2 euros, press B4, get a drink. No shopkeeper decides. The rules are built into the machine. A lending protocol is a vending machine that says *deposit collateral worth 100, take out at most 80 in loans*. A bug in the machine is a way to get drinks without paying, and on a blockchain, anyone in the world can try it.

Code is public, but not always readable

The blockchain stores a contract's **bytecode**: machine instructions, unreadable for humans. Developers write contracts in a language such as **Solidity**, then *compile* it to bytecode. To let people check what a contract does, a developer can publish the original source code. A service like **Sourcify** or Etherscan recompiles it and confirms it matches: the source is **verified**.

> **IN THE PROJECT'S CODE**
> `chain.is_verified()` asks Sourcify whether an oracle's source code is published. The app's *Oracle checks* table shows "source verified: yes / NO". In the PT-sUSDe example, the Morpho oracle `0xF8b2…` shows **NO**. That does not mean it is malicious, only that you cannot read what it does.

Functions and how the project calls them

A contract exposes **functions**. Some change things (`deposit`, `withdraw`), others only answer questions (`price()`, `owner()`, `asset()`). To call a function, you send its **selector**: the first 4 bytes of the fingerprint (keccak-256 hash) of its name. For example `price()` becomes `0xa035b1fe`. The answer comes back as a row of 32-byte "words" in hex, following a standard layout called the **ABI** (application binary interface).

Function	Selector	What the answer means
`price()`	`0xa035b1fe`	Morpho oracle: price of 1 collateral token in loan tokens.
`asset()`	`0x38d52e0f`	ERC-4626 vault: the token it holds (chapter 8).
`owner()`	`0x8da5cb5b`	Who owns (controls) the contract.
`SY()`	`0xafd27bf5`	Pendle PT: the wrapped token it redeems into (chapter 15).
`getThreshold()`	`0xe75235b8`	Safe multisig: how many signatures are needed.

> **IN THE PROJECT'S CODE**
> This is exactly what `src/graph/crawler.py` does for "Any contract" positions. It sends a contract every standard question at once. Every answer that is an address becomes a new **edge** in the graph, then it asks the same questions to that new address, up to 4 hops away and 40 contracts. Following links like this, from page to page, is called a **breadth-first crawl**.

Standards: ERC and EIP

An **EIP** (Ethereum Improvement Proposal) is a published design. An **ERC** (Ethereum Request for Comments) is an EIP that defines a standard interface. Standards make contracts predictable: every ERC-20 token has `balanceOf()`, every ERC-4626 vault has `asset()`. That predictability is what makes automatic dependency discovery possible.

Audits and hacks

An **audit** is a paid review of a contract's code by a security firm. It reduces risk but never removes it. Many hacked protocols were audited. A **hack** (or *exploit*) is when someone uses a flaw (a code bug, a manipulated price, a stolen admin key) to take money out of a contract.

> **REAL NUMBERS FROM THIS PROJECT**
>
> From `data/calibration.json`: over the last 3 years, DeFi protocols holding at least 100M USD suffered **22 hacks of 1M USD or more**. Across the 221 such protocols (after adjusting for hacks DeFiLlama could not link to a protocol), that is about **5.9% chance per protocol per year**, and a typical (median) hack took **4.1% of the protocol's money**.

> **CHECK YOURSELF**
>
> - What does "source verified" prove, and what does it not prove?
> - How does the crawler turn a contract's answers into graph edges?

7. Who controls a contract: proxies, owners, timelocks

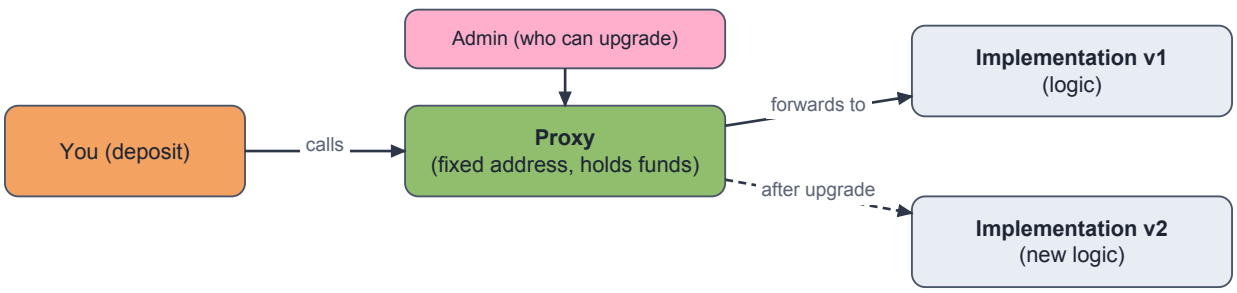
"Code is law" is only half true. Many contracts keep a **back door for their team**: the ability to change settings or even replace the code. That is useful for fixing bugs, and dangerous if the wrong person holds it. Knowing *who* holds it is a core part of dependency discovery.

Upgradeable contracts and proxies

Code stored on a blockchain cannot be edited. To make a contract upgradeable, developers split it in two:

- a **proxy**: the permanent address users interact with, which holds the money;
- an **implementation**: the contract with the actual logic, which the proxy forwards every call to.

An **admin** can point the proxy to a new implementation, that is, change the rules while keeping the money and the address. The standard **EIP-1967** says where the proxy stores these two addresses: in two fixed memory slots. The project reads those slots with `eth_getStorageAt` to learn who can upgrade.



An upgradeable contract: your money sits in the proxy. The admin can swap the logic behind it.

A **minimal proxy** (EIP-1167) is a cheap copy of another contract, used to deploy many identical vaults or oracles. It is *not* upgradeable. It just forwards. `chain.proxy_implementation()` recognises it from its code.

Roles you will see in the graph

Role	Plain meaning
owner	Can change the contract's settings, sometimes everything.
admin	Can upgrade the code (proxy admin).
curator	In a vault: decides which markets the money goes to and how much.
guardian	Can veto or pause dangerous changes (an emergency brake).
timelock	A mandatory delay between deciding a change and applying it.

IN THE PROJECT'S CODE

- In the graph these roles are **admin** nodes (pink), linked by `owned_by`, `administered_by`, `curated_by`, `guarded_by` or `controlled_by` edges.
- Morpho **Vault V2** (`src/graph/lending.py`) reads a timelock *per setting*. If the settings that decide where money can go (`addAdapter`, `increaseAbsoluteCap`, `increaseRelativeCap`) have **no timelock**, the curator could send deposits to a new market before depositors can react. The project flags this.
- Admin nodes carry no incident of their own. Key theft is already inside the hack rate of the contract they control, so counting it twice would overstate the risk.

REAL NUMBERS FROM THIS PROJECT

In the whole-book demo, the same **Safe 4/8** multisig (`0xbbac…f918`) and the same **Safe 4/9** (`0x21f7…73ca`) each control parts of **three different pockets**. If those signers were compromised, three pockets would be hit at once. Only a book-level view (chapter 26) can show this.

CHECK YOURSELF

- Why would a team make its contract upgradeable, and why does that create a risk?
- What does a timelock give depositors that an instant change does not?

8. Tokens

A **token** is a unit of value managed by a smart contract. Inside, the token contract is simply a table: *address* → *balance*. Sending tokens means asking the contract to lower one balance and raise another. Creating new tokens is **minting**. Destroying them is **burning**. Whoever is allowed to mint (the **minter**) can create value out of thin air, so the project looks for it (`minted_by` edges).

ERC-20: the standard token

Almost every token follows **ERC-20**: `balanceOf`, `transfer`, `symbol` ("USDC"), `decimals`. **Decimals** is how many digits are after the decimal point, because blockchains only store whole numbers. USDC has 6 decimals: 1 USDC is stored as 1,000,000. Most tokens have 18 decimals: 1 token is stored as 10^{18}.

WORKED EXAMPLE

To sell 1,000,000 USD of PT-sUSDe priced at about 0.993 USD, the project computes 1,000,000 / 0.993 ≈ 1,007,000 PT, then multiplies by 10^{18} before asking Pendle for a quote (`src/decision/exit_cost.py`, `pt_amount`). Forget the decimals and you ask for a quote on a trillion times the wrong amount.

Stablecoins

A **stablecoin** is a token designed to stay worth a fixed amount, usually 1 US dollar. Its **peg** is that target. When the price falls clearly below, it has **depegged**. How a stablecoin keeps its peg depends on what backs it:

Family	How it is backed	Examples	Depeg history (project data)
Fiat-backed	A company holds real dollars or treasury bills in a bank, 1 for 1.	USDC (Circle), USDT (Tether)	8.7% per coin-year, median drop 8.3%
Crypto-backed	Locked crypto worth more than the coins issued, or a hedged crypto position.	DAI, USDe (Ethena)	38% per coin-year, median drop 6.4%
Algorithmic	Mostly a mechanism and confidence, little hard backing.	UST (collapsed in 2022)	123% per coin-year, median drop 17.6%

"Per coin-year" means: if you held one such coin for one year, how many depeg episodes would you expect. Above 100% means more than one per year on average. The project defines a depeg as **trading below 0.97 for at least 2 days in a row**, for coins with at least 100M USD in circulation, since 2021.

Tokens that grow: yield-bearing tokens and vault shares

- **Yield-bearing token**: a token whose value in its base asset grows over time because the money behind it earns interest. **sUSDe** ("staked USDe") is worth slowly more and more USDe. The ratio is its **exchange rate**.
- **Vault share (ERC-4626)**: when you deposit in a standard vault, you receive *shares*. The vault's `asset()` function says which token it holds, and each share is a claim on a slice of it. The project uses `asset()` to look "inside" wrapped collateral, e.g. siUSD → iUSD.
- **Cooldown**: some tokens make you wait before redeeming (sUSDe has a cooldown period). The project reads `cooldownDuration()` and shows it in the exit cost explanation.

Wrapped and staked tokens

- **Wrapped token**: a token that represents another one 1:1 in a standard form. WETH is ETH wrapped as an ERC-20. WBTC is bitcoin represented on Ethereum (held by a custodian).
- **Staking**: locking ETH to help validate the chain, which earns rewards (chapter 2).
- **Liquid staking token (LST)**: a token you receive for staked ETH, so you can still use it (stETH from Lido, wstETH its wrapped form). It should be worth 1 ETH, but in June 2022 stETH traded about 5% below ETH. That is the basis of the project's JUDGEMENT *"asset_non_usd"* depeg row: 3% per year, 5% severity.

Real-world assets (RWA) and off-chain backing

An **RWA token** represents something outside the blockchain: a loan portfolio, a bond, euros in a bank. Examples in this project: **EURCV** (a euro stablecoin from Société Générale-FORGE), **PST** (from Huma, backed by payment-financing loans), **ONyc** (from OnRe, reinsurance). The blockchain can show the tokens, but not the assets behind them. You must trust the issuer's reports.

IN THE PROJECT'S CODE

When an issuer's DeFiLlama category means off-chain assets (RWA, RWA Lending, Basis Trading, CeDeFi, Off-Chain Credit), `add_offchain_backing()` adds an **"Off-chain backing"** box to the graph, flagged *"not verifiable on-chain"*. It marks **where the visible graph stops**. It carries no separate incident, because its risk is the issuer's.

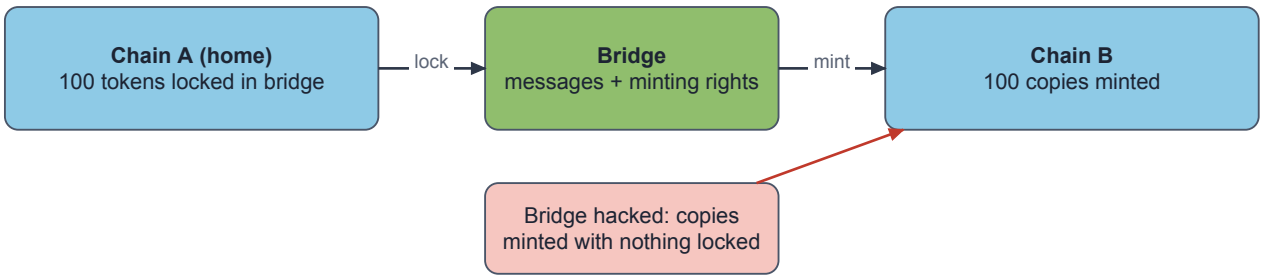
CHECK YOURSELF

- What does it mean for USDC to have 6 decimals?
- Why is a crypto-backed stablecoin more likely to depeg than a fiat-backed one, according to the data?
- What can the blockchain prove about an RWA token, and what can it not?

9. Bridges: moving tokens between chains

Chains are separate ledgers. Ethereum cannot see Solana's balances. To use a token on another chain, a **bridge** creates a representative copy there. Two common designs:

- **Lock and mint**: the original tokens are locked in a contract on chain A, and the bridge mints the same amount of copies on chain B. To go back, copies are burned and originals unlocked.
- **Burn and mint**: the token is burned on chain A and minted on chain B. The issuer authorises the bridge to mint on every chain.



A copy is only as good as the bridge. A bridge exploit can mint copies backed by nothing, and every lender that accepts the copy as collateral is exposed.

Bridges hold huge pools of locked tokens and have been some of the largest hacks in history (Ronin in 2022, more than 600M USD. Wormhole, about 320M USD).

The two bridge systems the project recognises

- **Chainlink CCIP** (Cross-Chain Interoperability Protocol): a registry contract on each chain (`TokenAdminRegistry`) maps a token to its **token pool**, the contract that locks/burns and mints it. The project asks the registry `getPool(token)`, then asks the pool which chains it is connected to.
- **LayerZero OFT** (Omnichain Fungible Token): a token that knows how to move itself across chains through a LayerZero **endpoint**. The project detects it by asking for `endpoint()` and `oftVersion()`.

REAL NUMBERS FROM THIS PROJECT

In the Huma USDC Main vault, the collateral **PST** on Ethereum is a **bridged copy**: minted by the CCIP pool `0xfd34…`, with its home on **Solana**. The project adds edges `PST → Chainlink CCIP (bridged_by)` and `PST → Solana (backed_on)`, and the flag: *"a bridge exploit mints tokens backed by nothing"*.

CHECK YOURSELF

- In lock-and-mint, where are the original tokens while the copies circulate?
- Why does a bridge hack hurt lenders on the destination chain?

PART 2

DeFi building blocks

Exchanges, oracles, lending markets, vaults, Pendle, Ethena and perpetual futures: the pieces a DeFi position is made of, and how they depend on each other.

10. DeFi, protocols and DeFiLlama

DeFi (decentralized finance) is the set of financial services built as smart contracts: exchanging, lending, borrowing, earning yield, trading with leverage. Anyone with a wallet can use them, at any hour, with no account opening. A **protocol** is one such service, usually a family of contracts with a name and a team: Pendle, Morpho, Aave, Kamino, Ethena, Hyperliquid…

TVL: the size of a protocol

TVL (total value locked) is the total value of the tokens deposited in a protocol's contracts, in USD. It is the standard measure of size. Pendle held about 1.2 billion USD in the project's cached data, Ethena's USDe about 4.8 billion.

DeFiLlama: the project's encyclopedia

DeFiLlama is a free public website and API that tracks thousands of protocols. The project uses four of its datasets:

Dataset	What it contains	Used for
Protocols	Name, **category** (Lending, Dexs, RWA, Basis Trading…), TVL, audits, past hacks.	Protocol and issuer nodes. The category picks the hack rate (chapter 20).
Yield pools	Every pool that pays a yield: chain, token symbol, TVL, underlying tokens.	**Issuer discovery**: who mints a token.
Hacks	Date, amount, technique of every recorded exploit.	Calibrating hack probabilities and severities.
Stablecoins	Daily prices and circulating supply of every stablecoin.	Calibrating depeg probabilities.

IN THE PROJECT'S CODE

How the project finds an issuer automatically (`defillama.find_issuer`): look at the *largest* yield pool of the token on that chain. If that pool **produces** the token from something else (Ethena turns USDe into sUSDe, Lido turns ETH into stETH) and is not a lender or exchange, it is the issuer. For USDC the largest pool is a lender, so no issuer is claimed rather than a wrong one. If that fails, the token's website on **CoinGecko** is matched to a DeFiLlama protocol with the same website (this is how PST is linked to Huma).

CHECK YOURSELF

- What does TVL measure?
- Why does the project require the *largest* pool before calling a protocol the issuer?

11. Exchanges and AMMs: where tokens are traded

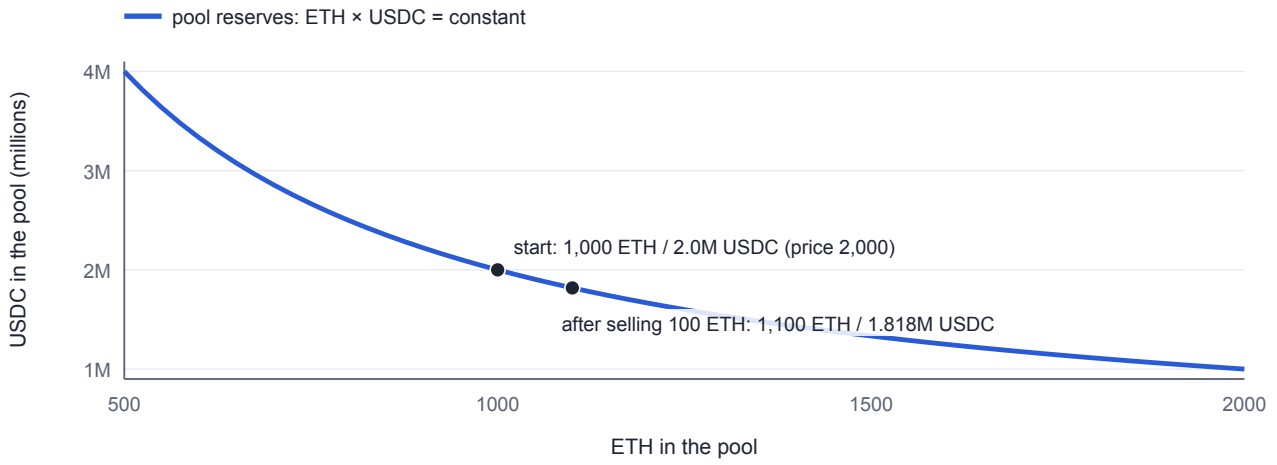
Centralized vs decentralized exchanges

- A **CEX** (centralized exchange) such as Binance, OKX or Bybit is a company. You send it your tokens, it keeps them (it has **custody**) and matches buyers and sellers in its own database. Fast and deep, but you trust the company. If it is hacked or fails (FTX in 2022), customers can lose their funds.
- A **DEX** (decentralized exchange) is a smart contract. You trade directly from your wallet against money sitting in the contract. Nobody holds your funds in between.

The AMM: a robot market maker

Most DEXes are **AMMs** (automated market makers). Instead of matching buyers and sellers, an AMM holds a **pool** of two tokens and quotes prices with a formula. People who deposit tokens into the pool are **liquidity providers (LPs)**. They earn the trading **fees**. The total value in the pool is its **liquidity**.

The simplest formula is the **constant product**: the pool keeps $x \times y = k$ constant, where x and y are its two reserves. When you sell token X, you add X to the pool, so the pool must give back enough Y to keep the product unchanged. The more you sell, the less each extra unit is worth.



Constant product: every trade moves the pool along this curve. The bigger the sale relative to the pool, the worse the average price.

WORKED EXAMPLE

A pool holds 1,000 ETH and 2,000,000 USDC, so k = 2 billion and the price is 2,000 USDC per ETH. You sell 100 ETH. The pool now has 1,100 ETH, so it must keep 2,000,000,000 / 1,100 = 1,818,182 USDC. You receive 2,000,000 − 1,818,182 = **181,818 USDC**, an average of 1,818 per ETH instead of 2,000. You lost **9.1%** to **price impact**.

Price impact (or **slippage**) is the gap between the price before the trade and the average price you actually get. It grows with the size of the trade relative to the pool. With reserves R on each side, selling an amount x (in value) costs:

$$\text{price impact} \approx x / (R + x)$$

IN THE PROJECT'S CODE

`amm_exit_cost()` in `src/decision/rule.py` is exactly this formula, with *R* = liquidity / 2. It is only a **fallback**. The project prefers a **live quote**: it asks the exchange *"how much would I get for this amount right now?"*.

For PT-sUSDe, the pool had 3.82M USD of liquidity. The formula would say selling 1M USD costs 1M / (1.91M + 1M) ≈ **34%**. Pendle's live router quote says **0.045%**. Pendle's curve is much flatter near the current price and the router also fills limit orders. Lesson: approximations can be wildly off, so live quotes matter.

Aggregators and quotes

A token often trades on many DEXes at once. An **aggregator** such as **KyberSwap** searches all of them and splits your trade across the best routes. Asking it for a **quote** (without trading) tells you the real price impact of selling a given amount today.

IN THE PROJECT'S CODE

`src/data/kyberswap.py` asks KyberSwap to quote selling a collateral token. The project first quotes a small sale (10,000 USD) to learn the token's price on DEXes. If even that small sale loses half its value or more (`NO_MARKET_IMPACT = 0.5`), the token has **no real market**. It then quotes selling what a 5% price drop would force liquidators to sell (chapter 13). The result is a **"DEX exit"** node in the graph.

CHECK YOURSELF

- Why does a large sale get a worse average price in an AMM?
- Recompute the example: what do you receive for selling 200 ETH?

12. Oracles: how contracts learn prices

A smart contract cannot look at a website. It only knows what is on its own chain. Yet a lending protocol must know what collateral is worth to decide when to liquidate. An **oracle** is a contract that publishes prices on-chain, so other contracts can read them.

- **Chainlink** is the largest oracle network. Independent node operators fetch prices from many exchanges, agree on a value and write it on-chain. A **price feed** (e.g. ETH/USD) is read through a **proxy** contract that forwards to an **aggregator**, the contract that actually stores the answers. That is why the project draws `reads_from` edges between oracle nodes.
- A **Morpho oracle** combines up to several feeds (`BASE_FEED_1`, `BASE_FEED_2`, `QUOTE_FEED_1`…) and sometimes a vault's exchange rate (`BASE_VAULT`) into one `price()`: the value of 1 collateral token in loan tokens, written with 36 extra digits of precision.

Market price vs reported rate

Some oracles do not report what the token trades at. They report a **rate** computed or declared by the issuer (*"1 PST = 1.0123 USDC"*). That is calm in normal times, but dangerous in a crisis: if the market price of the token collapses, the oracle does not move, so **liquidations never start**, and lenders keep lending against collateral that is worth less than they think.

REAL NUMBERS FROM THIS PROJECT

In the Huma USDC Main vault, the oracle of the PST/USDC market reads a feed called *"PST-USDC Exchange Rate (Calculated)"*. The project flags it: *"a reported rate, not a market price: a sell-off on markets does not move it, so liquidations do not start"*.

What can go wrong

- **Manipulation**: an attacker pushes the price on a thin market the oracle reads (often with a **flash loan**, a huge loan borrowed and repaid within one transaction), then borrows against overvalued collateral.
- **Stale price**: the feed stops updating while the market moves.
- **Wrong configuration**: the oracle reads the wrong feed or the wrong decimals.

IN THE PROJECT'S CODE

- `add_oracle()` in `src/graph/common.py` reads the oracle's price **on-chain** and compares it to the market price. The app's *Oracle checks* table shows the **gap to market** (−0.09% for PT-sUSDe: healthy) and whether the source is verified.
- Oracle failure is a **DATA** row: 3 oracle-manipulation hacks over 40 large price-consuming protocols in 3 years, about **4.4% per year**, median loss **3.5%** (`calibration.json`, key `_oracle`).

CHECK YOURSELF

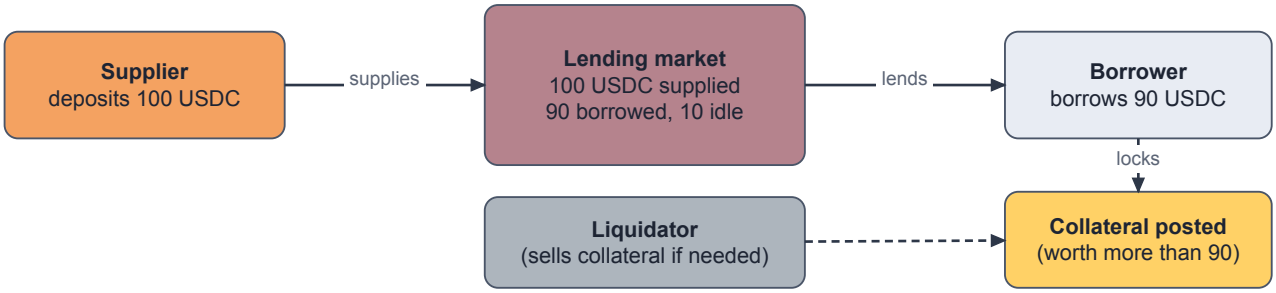
- Why can a lending protocol not simply "look up" a price?
- Why is an oracle that reports an issuer's rate risky for lenders in a crisis?

13. Lending markets: the heart of the project

Most positions in this project are, directly or indirectly, **loans**. Understanding lending well is the single most important step.

The basic deal

- **Suppliers** (lenders) deposit a token, say USDC, into the market and earn interest.
- **Borrowers** take USDC out. To do so they must first deposit **collateral** worth *more* than they borrow, for example PT-sUSDe or PST. This is **overcollateralization**: there is no credit check, the collateral is the only guarantee.
- If the collateral loses value, the borrower is **liquidated** before the loan becomes bigger than the collateral.

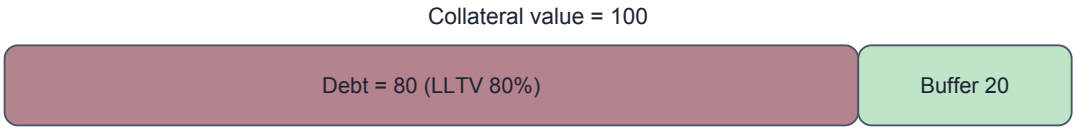


A lending market. Utilization here is 90 / 100 = 90%: only 10 USDC can be withdrawn right now.

LTV, LLTV and the safety buffer

LTV (loan-to-value) is loan ÷ collateral value. Borrow 80 against 100 and your LTV is 80%. Each market sets a maximum, the **LLTV** (liquidation LTV, Morpho's term) or **liquidation threshold** (Aave's term). Cross it and you can be liquidated.

For the lender, 1 − LLTV is a **buffer**. With LLTV 80%, a borrower at the limit owes 80 for 100 of collateral. The collateral can lose up to 20% and still cover the loan. Beyond that, the lender starts losing money, if the collateral cannot be sold in time.



A price drop eats the buffer first. Only beyond 20% does the lender lose.

Liquidation, the bonus, and bad debt

A **liquidation** is done by a **liquidator**, usually an automated bot. It repays part of the borrower's debt and receives the equivalent collateral **plus a bonus** (the **liquidation bonus** or *incentive*, typically 2 to 15%). It then sells that collateral, usually on a DEX. The bonus is its profit, and its reason to act fast.

This only works if selling the collateral costs the liquidator **less than the bonus**. If the DEX is too shallow and price impact exceeds the bonus, liquidating loses money, bots stop, and nobody repays the lender. When collateral ends up worth less than the debt, the difference is **bad debt**, and suppliers absorb it.

> **WORKED EXAMPLE**
> Morpho sets the bonus from the LLTV: bonus = 1 / (0.3 × LLTV + 0.7) − 1, capped at 15%. For the Huma market (LLTV 86%) that is 1 / 0.958 − 1 = **4.4%**. Selling PST collateral on DEXes returned **0%** of its value (no route exists). 100% impact is far above 4.4%, so liquidators would not step in, and the project marks the market's liquidation spiral as **"stalled"** (chapter 22).

Utilization: the number that decides if you can leave

$$\text{utilization} = \text{borrowed} \div \text{supplied}$$

Money that is lent out is not in the contract. A supplier can only withdraw the **idle liquidity** (supplied − borrowed). At 100% utilization, **nobody can withdraw** until borrowers repay or new suppliers arrive. This is the DeFi version of a bank run.

To avoid this, markets use an **interest rate model**: the borrow rate rises with utilization, slowly up to a **target** (about 90%), then steeply. High rates push borrowers to repay and attract new suppliers. Morpho uses an *adaptive* curve, Kamino a *kinked* one (two straight lines with a bend at the target).

> **KEY IDEA**
> For a lender, utilization measures two things at once. First, **how much of my money is exposed to the collateral**: only the lent-out part can be lost to bad debt. Second, **how easily I can leave**: only the idle part can be withdrawn now. That is why, in the project, the edge from a market to its collateral has **exposure = utilization**.

Three lending designs in the project

Protocol	Design	How the project models exposure
Morpho (Morpho Blue)	**Isolated markets**: each market is one loan token + one collateral token + one oracle + one LLTV, identified by a **market id**.	Market → collateral = utilization.
Aave v3	**Pools**: one pool, many tokens. Each supplied token can be borrowed against any accepted collateral (liquidation threshold per asset).	Pool → each collateral = utilization × that collateral's share of all collateral (top 8 kept).
Kamino (Solana)	**Markets made of reserves**: one reserve per token, borrowers may post any collateral reserve of the market. Like Aave.	Reserve → collateral = utilization × collateral share.

REAL NUMBERS FROM THIS PROJECT

The PT-sUSDe/USDC Morpho market: LLTV **91.5%**, 7.93M USD of PT as collateral, 6.99M USDC borrowed. Morpho's API (`marketCollateralAtRisk`) says how much collateral would be liquidated after each price drop: **0** at −2%, **3.79M** at −5%, **7.88M** at −10%. These forced sales feed the PT's exit risk (chapter 15) and the liquidation-spiral model (chapter 22).

CHECK YOURSELF

- With LLTV 90%, how far can collateral fall before the lender loses?
- Why would liquidators refuse to liquidate a borrower?
- A market has 50M supplied and 48M borrowed. What is its utilization, and how much can suppliers withdraw now?

14. Vaults and curators

Choosing among hundreds of lending markets is work. A **vault** does it for you: you deposit one token (say USDC), receive vault **shares**, and a professional **curator** spreads the money across several markets according to a strategy. You earn the blended interest minus a fee. The result is the **net APY** (annual percentage yield, after fees).

Morpho vaults

- **Vault v1 (MetaMorpho)**: the curator sets a **cap** per market (maximum amount) and a **supply queue** (the order in which new deposits fill markets). An **allocator** moves money between markets. A **guardian** can veto changes. A **timelock** delays risky changes (new markets, higher caps) so depositors can leave first.
- **Vault V2**: deposits go through **adapters**, plug-ins that connect the vault to a Morpho market, to another vault, or to other protocols. Caps can be **absolute** (USD) or **relative** (% of the vault). Each setting has its own timelock.

Kamino Earn vaults

The Solana equivalent: a Kamino **Earn vault** allocates deposits across Kamino **reserves**. Its liquidity is its idle tokens plus what each reserve can pay back right now.

KEY IDEA

A vault adds a **layer of people** (the curator, owner, guardian) on top of a layer of markets. The vault's exposure to a market is **its share of the vault**. Your exposure to a collateral is the product along the path: *vault share* × *market utilization*. And you can only withdraw what the vault can pull back from its markets.

REAL NUMBERS FROM THIS PROJECT

Huma USDC Main (Morpho vault): 100% allocated to one market, PST/USDC, with utilization **90.6%** and LLTV 86%. Vault size 12.84M USD, idle liquidity 3.59M USD, net APY **7.34%**. Owner and curator: the same **Safe 4/8**. Path: you → vault (100%) → market (100%) → PST (90.6%) → Huma → off-chain backing.

CHECK YOURSELF

- What does a curator decide?
- Why does a timelock on "add a new market" protect depositors?

15. Pendle: splitting yield from principal

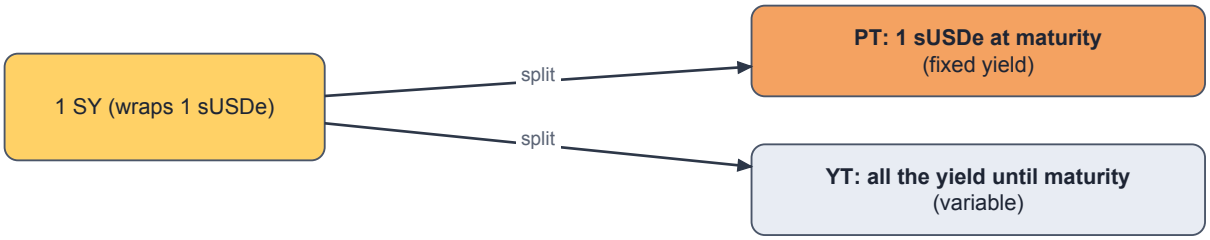
Pendle takes a yield-bearing token and splits it into two pieces that can be traded separately. To understand it, start with a familiar idea: a **zero-coupon bond**.

EVERYDAY ANALOGY

A zero-coupon bond pays no interest along the way. You buy it **below its face value** (say 95) and receive the full face value (100) at a fixed date. Your return is the discount. A Pendle **PT** is exactly that: pay a bit less than 1 today, receive 1 underlying token at maturity.

SY, PT and YT

- **SY** (standardized yield): a standard wrapper around a yield-bearing token (here sUSDe), so Pendle can handle any of them the same way.
- **PT** (principal token): redeemable **1:1 for the underlying at maturity** (here: 26 November 2026). Before that, it trades at a discount. Holding a PT to maturity locks in a **fixed yield**.
- **YT** (yield token): receives all the yield the underlying earns until maturity. It is a bet on future rates. The project does not hold YT.



Pendle splits a yield-bearing token into principal (PT) and yield (YT).

Discount and implied APY

If a PT costs p (in units of the underlying) and matures in d days, the yield you lock in, expressed per year, is the **implied APY**:

$$\text{implied APY} = (1 / p)^{365 / d} - 1$$

WORKED EXAMPLE

PT-sUSDe traded at a **0.61% discount** (p = 0.99385) with **46 days** to maturity. 1 / 0.99385 = 1.00618, raised to the power 365 / 46 = 7.93 gives 1.0501. Implied APY = **5.0%**, exactly what Pendle's API reported (`implied_apy` = 0.0501).

Selling before maturity: the Pendle AMM

Before maturity, the only way out is to sell the PT, mainly on **Pendle's own AMM pool** for that maturity. Its curve is designed for PTs: flat near the current rate, then steep. Pendle's **router** finds the best execution, including **limit orders** placed by other traders. That pool is your **exit door**, so its liquidity is a dependency.

PT as collateral, and why that matters to you

Many people deposit PTs as collateral on Morpho, borrow USDC, buy more PT, and repeat. This is **looping**, a way to multiply the fixed yield with leverage. If the PT price drops, these looped positions are liquidated, and liquidators **sell PT into the same Pendle pool** you would use to exit.

REAL NUMBERS FROM THIS PROJECT

A 5% PT price drop would liquidate **3.79M USD** of PT on Morpho. The Pendle pool had **3.82M USD** of liquidity. Ratio: 3.79 / 3.82 ≈ **0.99**. That is the `collateral_in` edge exposure of **99%** in the PT-sUSDe graph: in a crisis, forced sellers would be about as large as the whole exit pool.

CHECK YOURSELF

- What do you receive when a PT matures?
- Compute the implied APY of a PT at 0.98 with 182 days left.
- Why do liquidations on Morpho affect someone who never borrowed?

16. Ethena and USDe: a dollar built from hedges

USDe is a synthetic dollar issued by **Ethena**. It is not backed by dollars in a bank. It is backed by a **hedged crypto position**, also called a **delta-neutral** or **basis** trade.

Delta-neutral in one example

Ethena holds 1 ETH (worth 2,000 USD) and, at the same time, opens a **short** position of 1 ETH on a derivatives exchange (it gains when ETH falls). If ETH rises 10%, the ETH held gains 200 and the short loses 200. If ETH falls 10%, the reverse. Either way the total stays near 2,000 USD. That is "delta-neutral": no exposure to the price ("delta" = sensitivity to price).

Where does the yield come from? From staking rewards on the ETH, and above all from the **funding rate** that long traders usually pay short traders on perpetual futures (chapter 17). Users who stake USDe receive **sUSDe**, which collects that yield.

The dependencies this creates

- **Exchanges (venues)**: the short positions live on centralized exchanges. If an exchange is hacked or fails, the hedge breaks for the share of backing there.
- **Funding rates**: if they turn negative for long, the yield turns into a cost.
- **Custodians**: Ethena keeps collateral with off-exchange custodians, another dependency.

REAL NUMBERS FROM THIS PROJECT

Ethena's transparency API, read by `src/data/ethena.py`, shows where the backing sits. In the cached data: **Binance 13.3%** (638M USD), OKX 4.2%, Bybit 2.3%, Deribit 0.4%, CDE 0.2%. These become `hedges_on` edges from the Ethena issuer. The stress engine confirms the intuition: if Binance lost everything, the PT-sUSDe position would lose **13.3%**.

IN THE PROJECT'S CODE

Venues use the calibrated **exchange hack** row: 6 hacks of 10M USD or more on the 18 exchanges holding at least 1B USD over 3 years, about **11% per exchange per year**, median loss **2.6%** of holdings.

CHECK YOURSELF

- How can holding ETH and shorting ETH produce a stable dollar value?
- Why does a Binance hack matter to someone holding a Pendle PT?

17. Perpetual futures and Hyperliquid

Futures, perps, long and short

- A **future** is a contract to buy or sell an asset later at a price fixed today. A **perpetual future** ("perp") never expires.
- **Long** = you gain if the price rises. **Short** = you gain if it falls.
- **Margin** = the money you deposit as a guarantee. **Leverage** = position size ÷ margin. With 1,000 USD of margin and a 5,000 USD position, leverage is 5×.
- **Notional** = the full size of the position (5,000 USD here), not just the margin.
- **Funding rate**: a small payment exchanged every hour or so between longs and shorts, which keeps the perp price close to the real (spot) price. When more people want to be long, longs pay shorts.
- If losses eat the margin, the exchange **liquidates** the trader's position.

Hyperliquid

Hyperliquid is a perp exchange that runs on its **own blockchain** (Hyperliquid L1): an order-book trading engine (HyperCore) plus an EVM side (HyperEVM, chain 999). Its **validators** also act as the oracle: they publish prices built from major exchanges. Users deposit USDC through a **bridge contract on Arbitrum**.

A **Hyperliquid vault** lets you deposit USDC with a trader (the **leader**, who takes a commission) or with **HLP**, the exchange's own market-making and liquidation vault. Withdrawals have a **lock-up** after deposit: 4 days for HLP, 1 day for user vaults.

IN THE PROJECT'S CODE

`src/graph/perp_vault.py` builds: position → vault → **venue** Hyperliquid (100%) → Hyperliquid L1 chain. Venue → **bridge** on Arbitrum (custodies the USDC). Venue → **validator oracle**. Vault → each of its largest perp positions, with exposure = position notional ÷ vault value. This is the brief's *"vault deploying into an external venue"*.

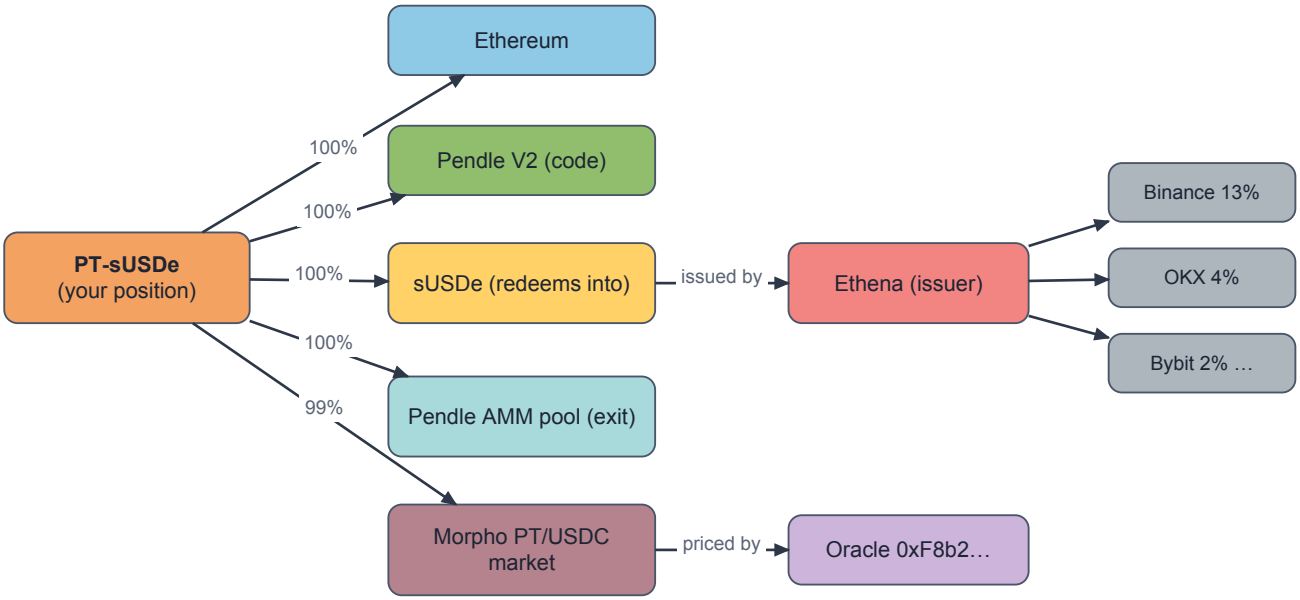
CHECK YOURSELF

- What is the difference between margin and notional?
- List three things a Hyperliquid vault depositor depends on.

18. Putting it together: the dependency map of a position

The hackathon brief describes a position's dependencies in **five layers**. Take PT-sUSDe and walk down them:

1. **Settlement chain**: Ethereum. If it halts, nothing moves.
2. **Yield source and issuer**: sUSDe, issued by Ethena, whose hedges sit on Binance, OKX…
3. **Oracle**: the Morpho oracle that prices PT-sUSDe for lenders.
4. **Market / AMM**: the Pendle pool where the PT can be sold before maturity.
5. **Lending markets**: Morpho markets where the PT is collateral, whose liquidations sell into that pool.



The real PT-sUSDe graph found by the project (simplified). Arrows mean "depends on". Percentages are edge exposures.

> **KEY IDEA**
> A position is never "just a token". It is a **web of dependencies**, and most of the risk sits in places you would never look at if you only watched the token's price. Part 3 turns this map into numbers.

PART 3

Risk: from a map to numbers

What can go wrong, how often, how badly, and how a failure deep in the graph travels up to your position.

19. Incidents: what can go wrong

The project describes every risk as a **scenario**: *node X suffers incident Y, with probability p within the horizon, and the position loses L if it happens*. There are five kinds of incident:

Incident	Plain meaning	Typical node
hack	Money stolen through a code bug, a manipulated price or a stolen key.	Protocols, issuers, vaults, exchanges
oracle_failure	The price feed is wrong, stale or manipulated.	Oracles
depeg	A token that should be worth 1 USD (or 1 ETH) trades clearly below.	Stablecoins, staked tokens
chain_halt	The blockchain stops producing blocks.	Chains
liquidity_crunch	Buyers vanish or withdrawals freeze: you cannot exit at a fair price.	AMM pools, lending markets, DEX exits

Three numbers per scenario

- **Annual rate**: how many such incidents to expect per year on this node (e.g. 0.059 = about one every 17 years).
- **Severity**: the fraction of *that node's* value lost when it happens (e.g. 4.1% of a protocol's funds).
- **Loss if hit**: the fraction of *your position* lost. This is not the same as severity: it depends on how much of your value flows through that node (chapter 22).

> **IN THE PROJECT'S CODE**
>
> `params_for(node)` in `src/risk/assumptions.py` picks the incident, rate and severity for each node type. Every row carries a **rationale** that starts with **DATA** (computed from history) or **JUDGEMENT** (an expert estimate, labelled as such). Admin nodes and "off-chain backing" boxes get no incident of their own, so the same risk is not counted twice.

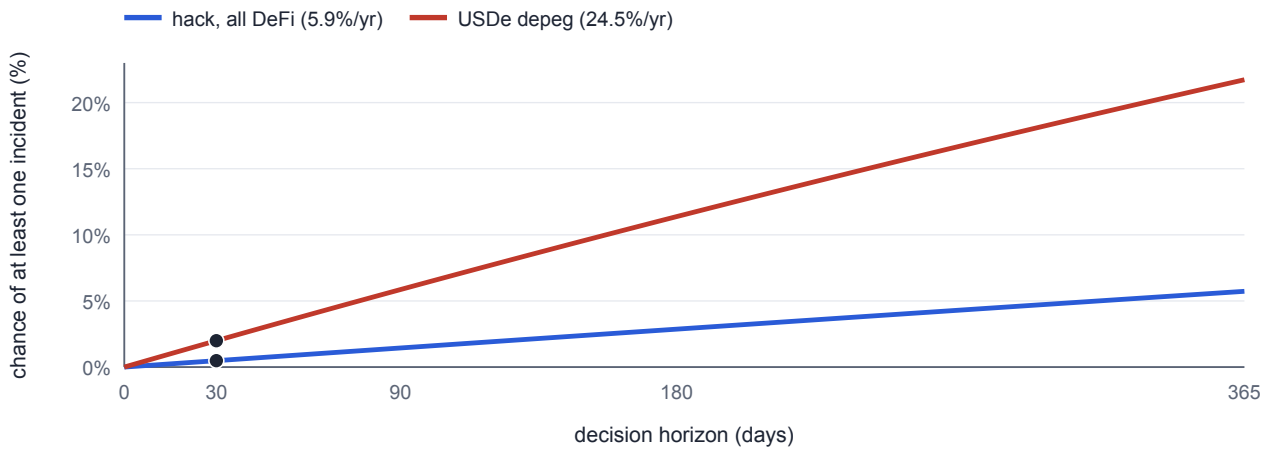
20. A probability toolkit

From an annual rate to a probability over your horizon

The decision is about a **horizon** (30 days by default). If incidents arrive at random at an average rate λ (lambda) per year, the chance of **at least one** within *d* days is given by the **Poisson model**:

$$P(\text{at least one incident in d days}) = 1 - e^{-\lambda \times d / 365}$$

For small numbers, this is close to λ × d / 365. But it never exceeds 100%, even for rates above one per year (some algorithmic stablecoins depeg more than once a year).



From an annual rate to the chance of an incident within the horizon (Poisson model). Dots at 30 days: 2.0% for a USDe depeg, 0.5% for a hack.

WORKED EXAMPLE

All-DeFi hack rate λ = 0.059 per year, horizon 30 days: $1 - e^{-0.059 \times 30 / 365} = 1 - e^{-0.00485} \approx$ **0.48%**.

Expected value

The **expected loss** of one scenario is probability × loss. A 2% chance of losing 6.4% is an expected loss of 0.128%. It is not what will happen. Most of the time nothing happens, and occasionally you lose 6.4%. It is what you lose **on average** if you could repeat the situation many times. Decisions under uncertainty compare averages like this.

Monte Carlo: rolling the dice 20,000 times

With a dozen scenarios that can happen together, exact formulas get messy. **Monte Carlo simulation** replaces maths with repetition: simulate the next 30 days 20,000 times. In each run, every scenario happens or not according to its probability (like a weighted coin), and the run's loss is computed. Then look at the 20,000 results.

Losses **compound** rather than add: if one incident takes 10% and another 20%, you keep 0.9 × 0.8 = 72%, so you lost 28%, not 30%:

$$\text{loss of a run} = 1 - \prod (1 - \text{loss}_i) \text{ over the incidents } i \text{ that happened}$$

- **Expected loss** = the average over all runs.
- **Worst-case loss (worst 5%)** = the average of the 5% worst runs. This tail measure, also called *expected shortfall*, answers "when it goes badly, how badly?"

IN THE PROJECT'S CODE

`simulate()` in `src/risk/model.py`: a 20,000 × (number of scenarios) table of random numbers, compared with the probabilities, gives which incidents hit in which run. `numpy` does it in one line. The random seed is fixed (`seed=0`), so the same input always gives the same output.

WATCH OUT

The single-pocket simulation assumes incidents are **independent**. In reality, a crisis often triggers several at once (a depeg, a liquidity crunch, an oracle lag). The whole-book view (chapter 26) partly fixes this by drawing a shared node's incident once for every pocket.

> **CHECK YOURSELF**
> - Compute the 30-day probability for a rate of 0.10 per year.
> - Why do losses of 10% and 20% compound to 28%?
> - What does "worst 5%" tell you that the expected loss does not?

21. Calibration: where the numbers come from

A risk number is only useful if it is **defensible**. The project's rule: every number carries a rationale, and wherever history exists, numbers come from data. Running `python -m src.risk.calibration` rebuilds `data/calibration.json` from public DeFiLlama data.

Hack rates per category

Method: count hacks of at least 1M USD, over the last 3 years, on protocols that held at least 100M USD just before the hack (the size of protocols an institutional book uses). Divide by the number of such protocols times 3 years. **Severity** = the median of (amount stolen − funds returned) ÷ TVL before the hack.

Category	Hacks per protocol per year	Median loss if hacked
All DeFi (`_all`)	5.9%	4.1%
Lending	9.1%	1.5%
Dexs	8.2%	29%
Bridge	6.6%	1.6%
Onchain Capital Allocator	5.6%	4.1% (all-DeFi)
Risk Curators (vaults)	2.8%	4.1% (all-DeFi)
RWA	2.4%	4.1% (all-DeFi)
Liquid Staking	1.4%	4.1% (all-DeFi)
Oracle manipulation (`_oracle`)	4.4%	3.5%
Large exchanges (`cex`)	11.1%	2.6%

Shrinkage: not trusting tiny samples

A category with 6 protocols and 0 hacks would get a 0% rate, which is obviously too optimistic. A category with 2 hacks among 3 protocols would get an absurd 22%. The project **pulls each category toward the all-DeFi rate**, as if 10 extra protocols with the average rate had been observed:

$$\text{rate} = (\text{hacks} + 10 \times 3 \text{ years} \times \text{all-DeFi rate}) \div (3 \text{ years} \times (\text{protocols} + 10))$$

A big category keeps mostly its own rate. A small one leans on the average. When fewer than 3 losses were measured, the all-DeFi severity is used. This is a classic statistical technique (a *Bayesian prior*, here as "pseudo-observations").

Depegs: blending a coin's own record with its family

For a stablecoin, the project counts **depeg episodes** (below 0.97 for 2+ days) per **coin-year** since 2021, per family (fiat-backed, crypto-backed, algorithmic). A coin's own record is blended with its family's, as if it had 5 more years at the family rate:

$$\text{rate} = (\text{own episodes} + 5 \times \text{family rate}) \div (\text{own coin-years} + 5)$$

WORKED EXAMPLE

USDe: **0 episodes in 2.75 coin-years**. Family: crypto-backed, **38% per year**. Rate = (0 + 5 × 0.38) ÷ (2.75 + 5) = 1.90 ÷ 7.75 ≈ **24.5% per year**. Over 30 days: $1 - e^{-0.245 \times 30/365} \approx$ **2.0%**. Severity: the crypto-backed median drop, **6.4%**. That is exactly the top row of the PT-sUSDe scenario table. Note how a young coin with a clean record still leans heavily on its family.

JUDGEMENT rows

Where no public dataset exists, the project uses labelled expert estimates:

Node	Incident	Rate / yr	Severity	Reasoning
Chain	chain_halt	1%	2%	Halts freeze but rarely destroy value.
Pendle pool (market)	liquidity_crunch	10%	3%	Exit pool thins, PT discount widens.
Lending market	liquidity_crunch	5%	10%	Liquidation cascade / bad debt.
DEX exit of a collateral	liquidity_crunch	10%	10%	DEX liquidity dries up, liquidators stop.
Contagion (other lender)	liquidity_crunch	5%	10%	Another lender's liquidations dump the same token.
Non-USD asset (LST)	depeg	3%	5%	stETH traded ~5% below ETH in June 2022.

KEY IDEA

Honest labelling matters as much as the numbers. A risk manager can challenge a JUDGEMENT row and replace it. A DATA row can be traced to the events it counts.

CHECK YOURSELF

- Why would a 0% hack rate for a small category be misleading?
- A coin has 1 depeg in 1 coin-year, family rate 8.7%. What blended rate does the formula give?

22. From the graph to your loss: exposure and the stress engine

Exposure along the graph

Each edge carries an **exposure** between 0 and 1: *what share of the source's value relies on the target*. Along a path, exposures **multiply**: if 100% of your money is in a vault, the vault puts 30% in market A, and market A has 90% utilization, your exposure to A's collateral is 1 × 0.3 × 0.9 = 27%. Over several paths, the simple version **adds** them and caps at 1.

IN THE PROJECT'S CODE

`path_exposure()` in `src/risk/model.py` uses networkx's `all_simple_paths` to list every path from the position to a node, multiplies along each path, sums, and caps at 1. It is shown as "linear exposure along the graph" in each scenario's rationale.

Why a smarter engine was needed

Multiplying exposures is linear: a 5% hit and a 50% hit produce proportional losses. Real finance is not linear. A lender loses **nothing** from a small collateral drop (the LLTV buffer absorbs it) and **a lot** from a large one. Liquidations can **amplify** a drop. And the same dependency reached through two paths should not be counted twice. `src/risk/stress.py` handles all three.

The propagation: dominoes, round after round

Hit one node with a shock (say "sUSDe loses 20%"). In each round, every node recomputes its loss from the losses of what it depends on. Repeat until nothing changes. Losses can only grow and are capped at 100%, so the process always settles (mathematicians call the end point a **fixed point**). This is the **DebtRank** method from research on bank contagion (Battiston et al., 2012). The existence of the fixed point comes from Eisenberg and Noe (2001).

Two kinds of edges

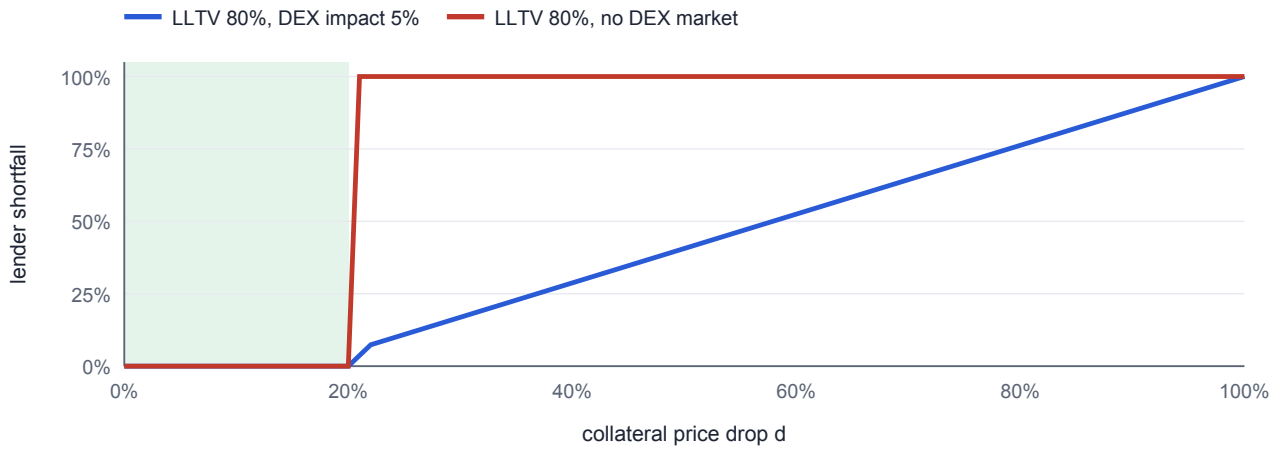
Edge type	Examples	How losses combine	Why
Value edges	`allocates_to`, `secured_by`, `holds`, `hedges_on`, `deposited_in`, `redeems_into`…	**Add up** (capped at 100%)	Each carries a different slice of the money. Losing 30% in market A and 30% in market B is 60%.
Common edges	`settles_on`, `runs_on`, `priced_by`, `issued_by`…	**Maximum**	They are dependencies of the *whole* value. Ethereum failing through 5 paths is still one failure.

Mechanism 1: the lender's buffer

For an edge from a lending market to its collateral (`secured_by`), the market loses only if the collateral drop *d* exceeds the buffer 1 − LLTV. Beyond it, the lender recovers the collateral, worth (1 − *d*), sold at a price impact *i*, against a debt of LLTV:

$$\text{shortfall} = 0 \text{ if } d \le 1 - \text{LLTV}, \text{ else } 1 - (1 - d)(1 - i) \div \text{LLTV}$$

and the market's loss = utilization × shortfall (only the lent-out part is at risk).



Nothing is lost inside the buffer (green). Beyond it, losses grow. With no DEX market (impact 100%), the lender cannot recover anything: a cliff.

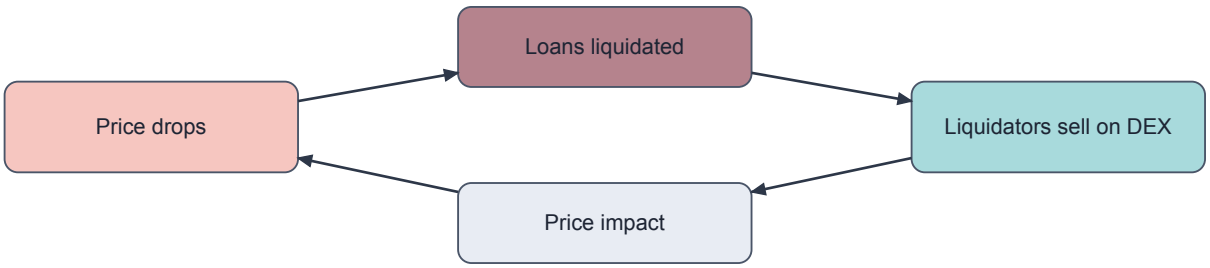
WORKED EXAMPLE

- LLTV 80%, drop 30%, impact 5%: 1 − 0.70 × 0.95 ÷ 0.80 = 1 − 0.831 = **16.9%** shortfall.
- LLTV 80%, drop 15%: inside the 20% buffer, so **0**.
- Huma market: LLTV 86%, PST drops 50%, no DEX market (*i* = 100%): 1 − 0.5 × 0 ÷ 0.86 = **100%** shortfall. × utilization 90.6% = the vault loses **90.6%**. This is the "haircut if trapped" of chapter 25.

Mechanism 2: the liquidation spiral

A price drop triggers liquidations. Liquidators sell the collateral, which pushes the price down further, which triggers more liquidations… The project measures one number per collateral, the **loop gain g**:

g = (price impact of the sales a 5% drop forces) ÷ 5%



The spiral. Each turn multiplies the extra drop by g.

Phase	Condition	What happens	Total drop from an initial drop d
Damped	g below 1	Each round adds less; the spiral fades.	d ÷ (1 − g) (geometric series)
Supercritical	g ≥ 1	The spiral feeds itself until forced sales run out.	1 − (1 − d)(1 − impact)
Stalled	impact above the liquidation bonus, or no DEX	Liquidators would lose money, so they stop. No spiral, but losses become bad debt.	d (and the buffer mechanism takes over)

WORKED EXAMPLE

A 5% drop forces sales that move the price 2% more: g = 2% ÷ 5% = 0.4 (damped). Total drop = 5% ÷ (1 − 0.4) = **8.3%**. The 1 + 0.4 + 0.16 + … series adds up to 1 / 0.6. If instead g = 1.2, each round is bigger than the last: supercritical.

The spectral radius ρ: one number for the whole system

Put all edge exposures in a table (a **matrix** W, one row and one column per node), with each collateral's loop gain on the diagonal. Linear algebra gives this matrix a **spectral radius** ρ (rho): roughly, *by how much the system multiplies a shock each round* when the shock travels along its loops. ρ ≥ 1 means a shock can feed itself (supercritical). On these graphs the only loops are spirals, so ρ is the largest loop gain. Think of it as the **echo factor** of the system.

REAL NUMBERS FROM THIS PROJECT

- PT-sUSDe: no measured spiral (the Pendle pool is the exit, not a DEX quote), ρ = 0. The stress table shows: Ethereum, Pendle, sUSDe or Ethena hit by X% → position loses X%. Morpho market or its oracle hit at 100% → **99.2%**. Binance hit at 100% → **13.3%**.
- Huma: PST's spiral is **stalled**. A 5% drop would force 12.7M USD of sales, plus 5% of the **183M USD** of PST held by other lenders, into DEXes that cannot absorb any of it, against a 4.4% bonus. So the risk shows up as bad debt, not as a spiral.

Contagion: other lenders holding the same collateral

If another protocol also accepts PST as collateral and liquidates, it sells into the **same** DEX liquidity. The project adds those protocols as **contagion** nodes (`also_collateral_in` edges), with exposure = their forced sales if 5% of their holding is liquidated ÷ the token's DEX liquidity (1 if the token has no market).

REAL NUMBERS FROM THIS PROJECT

PST in the Huma graph: contagion from Morpho (other markets, 0.91), Kamino on Solana (0.41), Fluid (0.37) and Jupiter Lend on Solana (0.10). Found from DeFiLlama, on Ethereum and on Solana where PST is bridged from.

CHECK YOURSELF

- Why do value edges add up but common edges take the maximum?
- Compute the shortfall for LLTV 90%, drop 25%, impact 10%.
- With g = 0.5, what does a 5% initial drop become?

23. Liquidity risk: can you get out, and at what price?

Losing value is one risk. **Not being able to leave** is the other. They are linked: the moment you most want to leave is the moment everyone else does too.

Leaving now: the exit cost

- **A PT**: sell it on Pendle. Cost = price impact + fee, from Pendle's **live router quote** for your exact size (0.045% for 1M USD of PT-sUSDe). If the router gives no quote (usually because the size is too large), the constant-product approximation is used.
- **A loan or vault**: withdrawing costs nothing (gas aside) **as long as there is idle liquidity**. Above it, the rest is stuck until borrowers repay.
- **A redeemable contract** (ERC-4626): redeem at the contract's own price, possibly after a cooldown.
- **Anything else**: no live quote, so the brief's reference cost of 11% is used.

Leaving late, with the crowd

The decision is taken **now**, before any incident. Staying means that *if* an incident hits, you will leave **late**, at the same time as everyone else. The project models that crowded exit:

- **PT**: you sell in the same batch as the PT that Morpho lenders liquidate after a 5% drop. But the price cannot fall forever. At some point, buyers who hold to maturity step in, because the PT still redeems 1:1. The project assumes they step in when holding to maturity earns **30% per year** (JUDGEMENT).
- **Loan or vault**: no price impact, but a **queue**. If every depositor runs, each gets at most idle liquidity ÷ total deposits.

WORKED EXAMPLE

PT-sUSDe: crowd = 3.79M USD of liquidated PT + your 1M = a 4.79M USD batch. The approximation says **71.6%** impact. But with 46 days to maturity, the 30%/yr floor is $1 - 1.30^{-46/365}$ = **3.25%**, so the late exit costs **3.25%**. Every incident costing more than 3.25% is increased by it: loss L becomes L + (1 − L) × 3.25%.

REAL NUMBERS FROM THIS PROJECT

Huma USDC Main: today **100%** of a 1M USD pocket can be withdrawn (3.59M USD idle). In a run where all 12.84M USD of deposits leave, each depositor gets **28%** out. The rest waits for borrowers to repay.

IN THE PROJECT'S CODE

`late_exit()` and `with_late_exit()` (`src/decision/exit_cost.py` and `src/decision/rule.py`). The app shows them side by side: *Leave now* vs *Leave late (after an incident, with the crowd)*.

CHECK YOURSELF

- Why can a PT's crisis price not fall below a certain level while its underlying holds?
- In a run, why do lenders not lose money to price impact, yet still suffer?

PART 4

Decisions: STAY or EXIT

Turning risk numbers into an answer a risk manager can act on, for one pocket and for the whole book.

24. The STAY / EXIT rule

The intuition: insurance

Leaving a position has a **certain** cost. Staying has an **uncertain** cost. The rule compares the two, on average:

EXIT if expected loss over the horizon > exit cost + yield given up

- **Expected loss**: from the Monte Carlo simulation (chapter 20), with each incident's loss computed by the stress engine and increased by the late-exit cost (chapter 23).
- **Exit cost**: today's quote to leave (slippage + fee for a PT, ~0 for a loan with idle liquidity).
- **Yield given up** (also called *forgone yield*): what you stop earning by holding cash instead. APY × days ÷ 365. With 6% APY over 30 days, that is 0.49%.

WATCH OUT

Why the yield given up matters: withdrawing from a vault costs ~0%. Without this term, any expected loss above zero said EXIT, for every loan. Leaving is never free: you lose the yield you came for. Adding it is what made the verdicts differ between vaults.

The breakeven probability p*

A risk manager often has their own view of how likely an incident is. The project gives them a threshold to compare it with:

p* = cost of leaving ÷ loss if hit (for the worst scenario)

WORKED EXAMPLE

The brief's reference case: exiting costs **11%** slippage. If the worst incident would cost the pocket **50%**, p* = 11 ÷ 50 = **22%**. If you believe the incident has more than a 22% chance of happening within the horizon, exit. Below that, stay. At the book level: with 10 pockets of 10%, unwinding one pocket at 11% costs 0.1 × 11% = **1.1% of the book**.

The exit trigger overrides the average

For lending positions there is a second, independent reason to leave: utilization has crossed the **exit trigger** u* (chapter 25), meaning withdrawals risk freezing with a large haircut. In that case the verdict is EXIT whatever the average says.

Real verdicts

REAL NUMBERS FROM THIS PROJECT

The six pockets of the whole-book demo, 1M USD each, 30-day horizon, computed by the latest code on the repository's cached snapshots:

Pocket	Expected loss	Cost of leaving (exit + yield)	Utilization / u*	Verdict
PT-sUSDe 26NOV2026	0.35%	0.46% (0.04% + 0.41%)	n/a (not a loan)	**STAY**
RockawayX USDC Yield	0.13%	0.52% (0 + 0.52%)	86.3% / 92.1%	**STAY**

Pocket	Expected loss	Cost of leaving (exit + yield)	Utilization / u*	Verdict
Huma USDC Main	0.18%	0.60% (0 + 0.60%)	90.6% / **84.3%**	**EXIT** (trigger)
Belem RWA EURCV	0.15%	0.22% (0 + 0.22%)	90.8% / **75.7%**	**EXIT** (trigger)
Kamino Steakhouse HY USDC	0.09%	0.33% (0 + 0.33%)	85.6% / 94.4%	**STAY**
Kamino RWA USDC	0.09%	0.49% (0 + 0.49%)	92.8% / **92.5%**	**EXIT** (trigger)

Notice that no pocket exits because of its *average* loss. The three EXITs all come from the freeze trigger: the risk that matters most for these vaults is **being stuck**, not a slow average loss.

The What if? control

In the app, *What if?* lets you pick any dependency and say how much it loses (e.g. "Binance loses 50%"). The stress engine immediately shows what your position loses, in % and USD. A second slider sets the **chance it happens within the horizon**. Above 0, the hit is added as an extra scenario, so the expected loss, the verdict and p* react to it. This is how a risk manager injects their own view (*"I think there is a 10% chance Ethena depegs 20% this month"*).

> **CHECK YOURSELF**
> - A loan pays 8% APY. What is the yield given up over 45 days?
> - Exit costs 2% and the worst incident costs 40%. What is p*?
> - Why can a pocket get EXIT even when its expected loss is below its cost of leaving?

25. When to leave a loan: the exit trigger

For a lender, the nightmare is not a small loss. It is a **freeze**: utilization hits 100%, withdrawals stop, and you are trapped while the collateral crashes. The question becomes one of **timing**: at what utilization should you leave, while you still can?

The ingredients

- **r**: the yield you earn per year by staying (the vault's net APY).
- **C**: the cost of leaving now (exit cost, usually ~0 for loans).
- **H**: the **haircut if trapped**: what you lose if you sit through a crisis. The stress engine computes it as your loss if your worst collateral falls **50%** (JUDGEMENT).
- **h(u)**: the **hazard** of a freeze, the expected number of freezes per year at utilization *u*. It explodes as *u* approaches 100%:

$$h(u) = \kappa \div (1 - u)^{\gamma} \text{ with } \gamma = 2, \text{ and } \kappa \text{ chosen so that } h(90\%) = 0.2 \text{ per year}$$

The anchor (JUDGEMENT): interest-rate curves keep markets near **90%** utilization on purpose, and a market sitting at that target is assumed to freeze about **once every five years** (0.2 per year). That gives $\kappa = 0.2 \times (1 - 0.9)^2 = 0.002$. At 95%, h = 0.8 per year. At 99%, h = 20 per year.

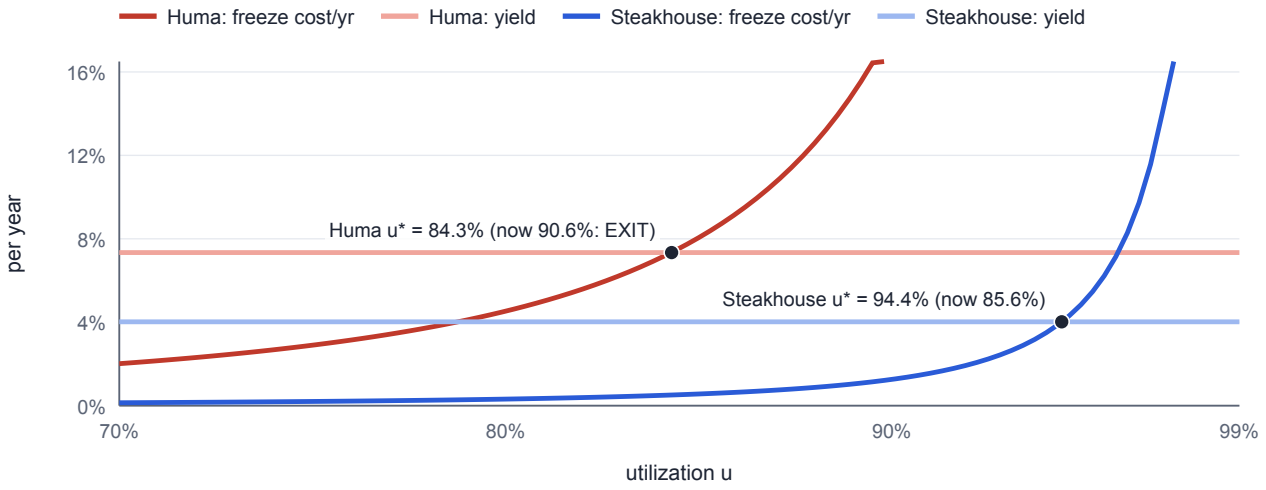
The rule: leave when one more day costs more than it pays

Over a short time, staying earns *r* × time. It also risks a freeze with probability *h(u)* × time, which would cost you *H* instead of the *C* you would pay by leaving now. So staying a little longer is worth it as long as:

$$r > h(u) \times (H - C)$$

Because the hazard only grows with utilization, this simple "one step ahead" comparison is the **optimal** rule (a result of optimal-stopping theory: the *Snell envelope*, solved here in closed form, per Chow, Robbins and Siegmund, 1971). Solving for *u* gives the threshold:

$$u^* = 1 - \left(\kappa \, (H - C) \div r \right)^{1/\gamma}$$



Exit when the expected cost of a freeze, h(u) × (H − C), overtakes the yield r. A big haircut H (Huma) moves the trigger far to the left.

WORKED EXAMPLE

Huma USDC Main: r = 7.34%, C = 0, H = **90.6%** (PST falls 50%, no DEX to sell it, so the lender recovers nothing on the 90.6% lent out). u* = 1 − √(0.002 × 0.906 ÷ 0.0734) = 1 − √0.0247 = 1 − 0.157 = **84.3%**. Current utilization **90.6%**: above the trigger, so **EXIT**. Freeze hazard now: 0.002 ÷ 0.094² ≈ 0.23 per year.

WORKED EXAMPLE

Kamino Steakhouse HY USDC: r = 4.02%, H = only **6.3%** (good collateral, sellable). u* = 1 − √(0.002 × 0.063 ÷ 0.0402) = 1 − 0.056 = **94.4%**. Now 85.6%: **STAY**. A small haircut means you can afford to wait much longer.

KEY IDEA

The trigger combines three facts into one actionable number. How bad is it to be trapped (H)? How well am I paid to wait (r)? How fast is the door closing (u)? A risk manager can monitor utilization live and act the moment it crosses u*.

CHECK YOURSELF

- Why does a bigger haircut H lower the trigger u*?
- Compute u* for r = 5%, H = 20%, C = 0.
- If H ≤ C, what does the project set u* to, and why?

26. The whole book: what pockets share

Belem manages a **book**: several pockets, e.g. 10 pockets of 10% each. Judging each pocket alone misses something crucial: pockets can **share dependencies**. Two vaults on Ethereum, both lending USDC through Morpho, controlled by the same multisig, are not two independent bets. One failure can hit both.

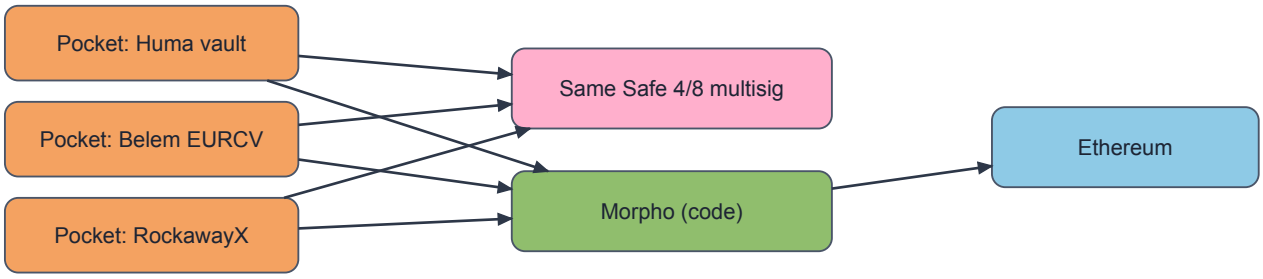
How the project finds shared dependencies

Node ids are the same across graphs: `chain:ethereum`, `asset:USDC`, `protocol:morpho-blue`, an admin's address… So after building each pocket's graph, the book simply counts in how many pockets each node id appears.

> **REAL NUMBERS FROM THIS PROJECT**
>
> Shared dependencies in the six-pocket demo book: **Ethereum** (4 pockets), **USDC** (4), **Morpho** (3), **Safe 4/8 multisig** (3), **Safe 4/9 multisig** (3), **Solana** (3), Kamino (2), reUSD (2), cbBTC (2), and an off-chain backing box (2). Even across chains: **PST** is collateral both behind Huma (on Ethereum, as a bridged copy) and behind Kamino RWA USDC (on Solana, its home).

Correlated vs independent: the book Monte Carlo

The book simulation draws **each incident once** and hits every pocket that depends on that node in the same run. For comparison, it also runs the naive version, where every pocket draws its own incidents, as if the pockets were independent. The difference is the risk a pocket-by-pocket view **hides**.



Real, from the demo book: Huma, Belem RWA EURCV and RockawayX share the same Safe 4/8 multisig and Morpho. One compromised multisig or one Morpho bug hits all three.

> **REAL NUMBERS FROM THIS PROJECT**
>
> Six pockets of 1M USD, 30 days. Expected book loss: **10,111 USD** with shared draws vs 9,786 USD if independent (averages barely change). **Worst 5%**: **116,707 USD** with shared draws vs **100,911 USD** if independent. Pretending the pockets are independent **hides about 15,800 USD of tail risk (+16%)**. Shared dependencies do not change the average much, but they make bad days much worse.

What the Whole book view shows

- A **headline**: how many pockets should EXIT, the book size, the expected and tail losses, and the hidden tail.
- **Pocket cards**: verdict, expected loss, cost of leaving, a one-line reason and the **action** (e.g. *"Withdraw all 1.00M USD now (no slippage)"* or *"Sell the PT on Pendle now"*).
- A **shared-dependency heatmap** (which pocket relies on which node) and a **network** picture of the book.
- A **book stress test**: pick a node and a severity, see each pocket's loss.

- An **action plan** listing what to do pocket by pocket.
- An editable table to build your own book: add a row, paste an address.

WATCH OUT

The book simulation still treats **different** nodes as independent. A market-wide crash that hits Ethena, Morpho and Pendle at once is not modelled as one event. CLAUDE.md lists this as a known gap.

CHECK YOURSELF

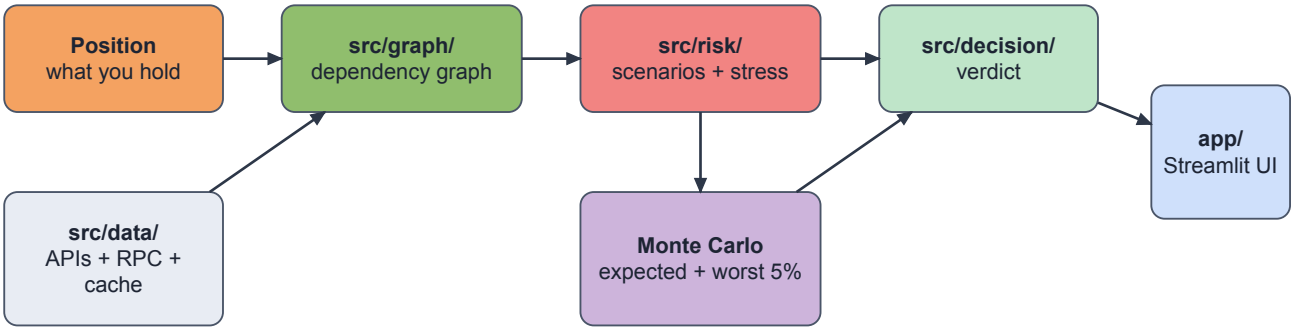
- Why can the average book loss stay almost the same while the worst 5% rises?
- Name two kinds of dependency that two pockets on different chains could still share.

PART 5

The project's code

How the concepts of Parts 1 to 4 map onto files, functions and screens.

27. The pipeline, file by file



The pipeline: a position goes in, a STAY / EXIT verdict with its reasons comes out.

The data contract: src/schemas.py

Every module talks through a few shared objects. Think of them as the forms that pass from desk to desk:

Object	Fields (simplified)	Concept
`Position`	chain_id, address, size_usd, horizon_days, book_share, kind	What you hold. `kind`: pendle_pt, morpho_supply, aave_supply, morpho_vault, hyperliquid_vault, contract, kamino.
`Node`	id, type, name, chain, meta	One dependency. `type` is the box colour in the app.
`Edge`	source, target, kind, exposure, evidence	"source depends on target". Every edge must carry evidence.
`DependencyGraph`	position_id, nodes, edges	The map of chapter 18.
`Scenario`	node_id, incident, probability, loss_if_hit, rationale	One line of the risk table (chapter 19).
`RiskResult`	scenarios, expected_loss, worst_case_loss	Monte Carlo output.
`Verdict`	decision, expected_loss, exit_cost, reason, breakeven_probability	The answer (chapter 24).

Folder by folder

File	What it does	Chapters
`src/data/cache.py`	Every HTTP call goes through `fetch_json`, saved to `data/cache/`; falls back to it if the API is down.	4
`src/data/chain.py`	JSON-RPC reads (`eth_call`, `eth_getCode`…) and Sourcify checks.	4, 6
`src/data/pendle.py`, `morpho.py`, `aave.py`, `kamino.py`, `hyperliquid.py`, `ethena.py`	One fetcher per protocol API.	13-17
`src/data/defillama.py`, `coingecko.py`, `kyberswap.py`	Protocols, yield pools, hacks; token websites; DEX quotes.	10, 11
`src/graph/build.py`	Picks the builder for `Position.kind`.	27
`src/graph/pendle_pt.py`	The five layers of a PT.	15, 18

File	What it does	Chapters
`src/graph/lending.py`	Morpho market, Aave pool, Morpho vault v1 and V2.	13, 14
`src/graph/kamino.py`	Kamino Earn vaults and reserves (Solana).	13, 14
`src/graph/perp_vault.py`	Hyperliquid vaults.	17
`src/graph/crawler.py`	Any contract: standard questions, breadth-first, controllers.	6, 7
`src/graph/collateral.py`	Behind each collateral: issuer, bridge, controllers, DEX exit, contagion, unwrapping.	8, 9, 11, 22
`src/graph/common.py`	Chains, explorers, issuer discovery, oracle checks, off-chain backing.	5, 10, 12
`src/risk/calibration.py`	Builds `data/calibration.json` from history.	21
`src/risk/assumptions.py`	Rate and severity per node (DATA / JUDGEMENT).	19, 21
`src/risk/stress.py`	Propagation, buffer, spiral, spectral radius.	22
`src/risk/model.py`	Scenarios, Poisson, path exposure, Monte Carlo.	20, 22
`src/decision/exit_cost.py`	Exit now (live quote) and late exit (with the crowd).	23
`src/decision/rule.py`	STAY / EXIT, p*, late-exit adjustment.	24
`src/decision/snell.py`	The exit trigger u*.	25
`src/decision/book.py`	Whole book: evaluation per pocket, shared deps, book Monte Carlo.	26
`app/main.py`, `app/book_view.py`	The Streamlit app: one pocket / whole book.	28

One position, step by step in code

```
pos     = Position(1, "0x47ad…ef77e", 1_000_000, horizon_days=30, kind="pendle_pt")
graph   = build_graph(pos)                        # src/graph: nodes + edges with evidence
late    = late_exit(pos, graph)                   # cost of leaving with the crowd
scen    = with_late_exit(build_scenarios(graph, 30), late["cost"])
risk    = simulate(scen)                          # Monte Carlo: expected, worst 5%
cost, _ = exit_cost(pos, graph)                   # live Pendle quote
rule    = exit_rule(graph, cost)                  # u* (None for a PT)
forgone = position_yield(graph) * 30 / 365        # yield given up
verdict = decide(risk, cost, forgone, rule)       # STAY / EXIT + reason
```

This is `book.evaluate()` in `src/decision/book.py`. The app's single-pocket view does the same, step by step, showing each intermediate result.

28. Running and reading the app

Commands

```
pip install -r requirements.txt          # install the libraries
streamlit run app/main.py                # the app, live data (cache as fallback)
DEFI_OFFLINE=1 streamlit run app/main.py # demo from data/cache only (no internet needed)
pytest                                   # tests (graph tests run offline)
python -m src.risk.calibration           # rebuild data/calibration.json
python -m src.trace morpho_vault 1 <vault> --api morpho  # API calls behind a graph
```

One pocket view, top to bottom

1. **Sidebar**: choose the view (*One pocket* / *Whole book*), the pocket type, chain, market or vault, size, horizon and share of book.
2. **1. Dependency graph**: coloured boxes (legend below it: colours are *types*, not risk levels), arrows labelled with edge kind and exposure. Expanders: *Edges and evidence* (every link and its proof), *API calls behind this graph* (what was asked, live or cache).
3. **Red flags**: single-wallet owners, low-threshold multisigs, missing timelocks, unpublished code, bridged copies, reported-rate oracles, no DEX market…
4. **Oracle checks**: on-chain price, gap to market, source verified, proxy.
5. **2. Incident scenarios**: one row per node and incident, with probability, loss if hit, expected loss and the full rationale (DATA / JUDGEMENT).
6. **Stress engine**: *What if?* (pick a dependency, how much it loses, and optionally its chance), then the table of every dependency hit at 5 / 20 / 50 / 100%, the spectral radius ρ and each collateral's spiral phase.
7. **3. Decision**: verdict, expected loss, worst 5%, exit cost, p*, yield given up; *Leave now* vs *Leave late*; the *Exit trigger* panel (utilization, u*, haircut, hazard).

> **KEY IDEA**
> Read the app like an argument. **What do I depend on** (graph and evidence), **how likely is each failure and how bad for me** (scenarios and stress), **so what do I do** (decision). Each number has a rationale you can click into and challenge.

29. Honest limits and where to go next

A strong risk tool says what it does **not** know. From the code and CLAUDE.md:

- **Coverage**: Kamino oracles and Solana program authorities are not read. Euler is not covered (no keyless API). Venue breakdown exists only for Ethena. Aave PT exposure is still crude (share of PT TVL).
- **JUDGEMENT anchors**: chain halts, Pendle pool crunches, lending cascades, the 30%/yr crisis PT yield, the freeze hazard (κ, γ) and the 50% crisis drop are expert estimates. They are labelled, but they move results.
- **Correlation**: within a pocket, incidents are independent draws. In the book, different nodes are independent. A market-wide crash hitting many nodes at once is not one event yet.
- **Late exit**: ignores liquidity providers pulling their liquidity from the Pendle pool in a crisis, which would make the crowded exit worse.
- **Constant-product fallback**: can be far from Pendle's real curve (chapter 11); live quotes are used whenever available.
- **Snapshots**: numbers in this course come from cached data in the repository. Live runs will differ.

> **KEY IDEA**
> You now have every concept the project uses: ledgers and contracts, tokens and bridges, AMMs, oracles, lending, vaults, Pendle, Ethena and perps; incidents, probabilities and calibration; propagation, buffers and spirals; exit costs, optimal stopping and portfolio correlation. The best next step is to open the app, pick a pocket, and explain every number on the screen out loud using this course.

PART A

Appendices

Formula sheet, glossary, and answers to the *Check yourself* questions.

A. Formula sheet

Quantity	Formula	Where
Utilization	borrowed ÷ supplied	Ch. 13
Implied APY of a PT	$(1 / p)^{365/d} - 1$	Ch. 15
AMM price impact (fallback)	x ÷ (R + x), R = liquidity / 2	Ch. 11
Morpho liquidation bonus	1 ÷ (0.3 × LLTV + 0.7) − 1, max 15%	Ch. 13
Horizon probability (Poisson)	$1 - e^{-\lambda\, d\, /\, 365}$	Ch. 20
Loss of a Monte Carlo run	$1 - \prod (1 - \text{loss}_i)$	Ch. 20
Hack rate with shrinkage	$(n + 10 \times 3 \times \text{rate}_{all}) \div (3 \times (N + 10))$	Ch. 21
Depeg rate blend	$(\text{episodes} + 5 \times \text{rate}_{family}) \div (\text{coin-years} + 5)$	Ch. 21
Path exposure	Σ over paths of ∏ edge exposures, capped at 1	Ch. 22
Lender shortfall	0 if d ≤ 1 − LLTV, else 1 − (1 − d)(1 − i) ÷ LLTV	Ch. 22
Spiral loop gain	g = impact of forced sales at −5% ÷ 5%	Ch. 22
Damped spiral total drop	d ÷ (1 − g)	Ch. 22
Crisis PT floor (late exit)	$1 - 1.30^{-\text{days}/365}$	Ch. 23
Late-exit adjusted loss	L + (1 − L) × late cost, if L ≥ late cost	Ch. 23
Yield given up	APY × days ÷ 365	Ch. 24
Decision	EXIT if expected loss > exit cost + yield given up, or u ≥ u*	Ch. 24
Breakeven probability	p* = cost of leaving ÷ worst loss if hit	Ch. 24
Freeze hazard	$h(u) = \kappa \div (1 - u)^{\gamma}$, γ = 2, κ = 0.002	Ch. 25
Exit trigger	$u^* = 1 - (\kappa\,(H - C) \div r)^{1/\gamma}$	Ch. 25

B. Glossary

ABI: Application binary interface: the standard layout of a contract call and its answer (32-byte words in hex).

Adapter: In a Morpho Vault V2, a plug-in connecting the vault to a market, another vault or another protocol.

Address: Public identifier of a wallet or contract (0x + 40 hex characters on EVM chains).

Admin: Address allowed to upgrade a contract or change its settings. Pink nodes in the app.

Aggregator (DEX): Service that searches many DEXes for the best price (KyberSwap).

Aggregator (oracle): The Chainlink contract that stores price answers, behind a proxy.

AMM: Automated market maker: a DEX that prices trades with a formula over a pool of tokens.

APY: Annual percentage yield: yearly return including compounding. *Net APY* = after fees.

Audit: Paid security review of a contract's code. Reduces but never removes risk.

Bad debt: Loans whose collateral is worth less than the debt. Suppliers absorb the difference.

Basis trade: Holding an asset and shorting its future to earn the funding / price difference without price exposure (Ethena).

Block: A batch of transactions added to the chain together.

Block explorer: Website showing a chain's transactions and contracts (Etherscan, Solscan…).

Book: All the pockets a fund manages together.

Breakeven probability (p*): Incident probability above which exiting is cheaper than staying.

Bridge: System that moves tokens between chains (lock-and-mint or burn-and-mint).

Burn: Destroy tokens, reducing the supply.
Bytecode: The machine code of a contract as stored on-chain.
CCIP: Chainlink's Cross-Chain Interoperability Protocol, a bridge standard.
CEX: Centralized exchange (Binance, OKX…): a company holding customers' funds.
Chain halt: The chain stops producing blocks. Funds are frozen until it restarts.
Chain ID: Number identifying an EVM chain (1 = Ethereum).
Collateral: Tokens a borrower locks as a guarantee for a loan.
Consensus: The rules by which validators agree on the next block.
Constant product: AMM rule x × y = k.
Contagion: A failure spreading to others through shared exposures, e.g. lenders dumping the same collateral.
Cooldown: Waiting period before a token can be redeemed.
Curator: Manager deciding where a vault's money goes.
DebtRank: Method to spread losses through a network of exposures, round by round, capped at 100%.
Decimals: How many digits after the decimal point a token uses (USDC: 6, most: 18).
DeFi: Decentralized finance: financial services built as smart contracts.
DeFiLlama: Public data site and API on protocols, TVL, yields, hacks and stablecoins.
Delta-neutral: A position whose value does not move with the price of the asset.
Depeg: A token trading clearly below its target (project: below 0.97 for 2+ days).
DEX: Decentralized exchange: a smart contract where tokens are traded from wallets.
DEX exit: The project's node for whether liquidators can sell a collateral on DEXes.
EIP / ERC: Ethereum improvement proposal / a standard interface (ERC-20, ERC-4626).
EIP-1967: Standard storage slots where a proxy keeps its implementation and admin.
EOA: Externally owned account: a wallet controlled by one private key.
ERC-20: The standard token interface.
ERC-4626: The standard vault interface: deposit a token, get shares; `asset()` names the token.
Evidence: URL or note proving a dependency edge exists. Required on every edge.
EVM: Ethereum Virtual Machine: the computer running smart contracts on Ethereum and compatible chains.
Exit cost: What it costs to leave a position now (slippage + fees).
Expected loss: Probability-weighted average loss.
Expected shortfall: Average of the worst outcomes (here the worst 5% of runs).
Exposure: Share of a node's value that relies on another node (edge weight, 0 to 1).
Finality: The point after which a block cannot be undone.
Fixed point: State where another round of propagation changes nothing.
Flash loan: A loan borrowed and repaid within one transaction, often used in oracle manipulation.
Forgone yield: Yield given up by leaving a position: APY × days ÷ 365.
Funding rate: Periodic payment between longs and shorts that keeps a perp near the spot price.
Gas: Fee paid to validators to execute a transaction.
Guardian: Role that can veto or pause changes in a vault.
Haircut (H): Loss if trapped through a crisis (stress engine, collateral −50%).
Hash: Digital fingerprint of data. Any change gives a different hash.
Hazard: Expected number of events per year at a given state (freeze hazard h(u)).
Hex: Base-16 notation (0-9, a-f), prefixed 0x.
HLP: Hyperliquid's own market-making and liquidation vault.
Horizon: The period the decision covers (30 days by default).
Idle liquidity: Supplied tokens not lent out: what can be withdrawn now.
Implementation: The logic contract behind a proxy.

Implied APY: Fixed yield locked in by buying a PT at its current price.

Interest rate model: Formula setting borrow rates from utilization.

Issuer: The entity or protocol that creates a token and manages its backing.

JSON-RPC: Standard message format to ask a blockchain node questions.

JUDGEMENT: Label for an expert estimate where no data exists.

Leverage: Position size ÷ own money (margin).

Limit order: An order to buy or sell at a set price, filled by Pendle's router when it is better.

Liquidation: Forced closing of a loan whose collateral fell too far; the liquidator repays and takes collateral plus a bonus.

Liquidation bonus: Extra collateral a liquidator receives; its profit margin.

Liquidity: How much can be traded or withdrawn without moving the price; value in a pool.

Liquidity provider (LP): Someone depositing tokens into an AMM pool to earn fees.

LLTV: Liquidation loan-to-value: the maximum loan ÷ collateral before liquidation (Morpho).

Lock-up: Period after deposit during which you cannot withdraw.

Long / short: Gaining when the price rises / falls.

Looping: Repeatedly borrowing against a token to buy more of it (leverage).

LST: Liquid staking token, e.g. stETH: a tradable claim on staked ETH.

LTV: Loan-to-value: loan ÷ collateral value.

Margin: Money deposited to back a leveraged position.

Market id: Identifier of a Morpho market (loan token, collateral, oracle, LLTV).

Maturity: Date when a PT redeems 1:1 for its underlying.

Minimal proxy: EIP-1167 clone: a cheap, non-upgradeable copy forwarding to another contract.

Mint: Create new tokens.

Monte Carlo: Estimating outcomes by simulating many random scenarios.

Multisig (Safe): Contract wallet needing k of n signatures.

Node (blockchain): Computer keeping a full copy of the ledger and answering RPC calls.

Node (graph): One dependency in the project's graph.

Notional: Full size of a derivatives position.

Off-chain backing: Assets outside the blockchain (loans, bonds, bank deposits) behind a token; where the visible graph stops.

OFT: LayerZero Omnichain Fungible Token: a token that moves itself across chains.

Oracle: Contract publishing prices on-chain for other contracts.

Overcollateralization: Requiring collateral worth more than the loan.

Peg: The fixed value a stablecoin targets (usually 1 USD).

Perpetual future (perp): A futures contract with no expiry.

Pocket: One slice of the book, one position (e.g. 10% of the fund).

Poisson model: Model of random events at a constant rate; gives 1 − e^(−rate × time).

Pool: Tokens deposited in a contract for trading (AMM) or lending (Aave).

Price impact / slippage: How much worse your average price is than the starting price, due to your trade's size.

Private key: Secret number that controls an address.

Proof of stake: Consensus where validators lock money as a guarantee of honesty.

Protocol: A DeFi service made of smart contracts.

Proxy: Permanent contract address forwarding calls to a replaceable implementation.

PT: Pendle principal token: redeems 1:1 for the underlying at maturity.

Quote: The price an exchange would give for a trade now, without trading.

Reserve: In Kamino or Aave, the lending pool of one token.

Router: Contract that finds the best way to execute a trade (Pendle's router).
RPC: Remote procedure call: asking a node to run a function and return the answer.
RWA: Real-world asset: token representing something off-chain.
Scenario: Node X suffers incident Y with probability p and costs the position L.
Seed phrase: 12 or 24 words encoding a private key.
Selector: First 4 bytes of the hash of a function's name, used to call it.
Severity: Fraction of a node's value lost in an incident.
Shares: Units representing your slice of a vault.
Shrinkage: Pulling a small sample's estimate toward a broader average.
Signature: Proof a transaction was approved by the private key's owner.
Smart contract: Program on a blockchain that holds money and follows its code.
Snell envelope: Optimal-stopping tool: the value of choosing the best time to stop.
Sourcify: Service verifying that published source code matches on-chain bytecode.
Spectral radius (ρ): Largest eigenvalue size of the exposure matrix; ≥ 1 means a shock can feed itself.
Spiral (liquidation): Liquidations push the price down, causing more liquidations.
Stablecoin: Token designed to keep a fixed value (fiat-backed, crypto-backed, algorithmic).
Staking: Locking tokens to help secure a chain, for rewards.
Stress test: Hitting a node with a given loss and measuring the effect on the position.
Supply queue: Order in which a Morpho vault fills its markets with new deposits.
sUSDe: Staked USDe: the yield-bearing version of Ethena's synthetic dollar.
SY: Pendle standardized yield: wrapper around a yield-bearing token.
Timelock: Mandatory delay between deciding and applying a change.
Token: Unit of value managed by a smart contract.
Transaction: Signed instruction that changes the ledger.
TVL: Total value locked: USD value deposited in a protocol.
u* (exit trigger): Utilization above which a lender should leave before a freeze.
USDe: Ethena's synthetic dollar backed by a delta-neutral position.
Utilization: Borrowed ÷ supplied in a lending market.
Validator: Computer that proposes and confirms blocks, with money at stake.
Vault: Contract pooling deposits and allocating them according to a strategy.
Venue: External trading place (an exchange) a protocol depends on.
Wrapped token: Token representing another one 1:1 in a standard form (WETH, WBTC).
YT: Pendle yield token: receives the underlying's yield until maturity.

C. Answers to Check yourself

Chapter 1

1. Because money already is a ledger entry. A blockchain is a way to keep that ledger without trusting a single keeper.
2. Spending the same digital money twice. The bank keeps the single official ledger and refuses the second payment.

Chapter 2

1. Block 99's fingerprint changes, so the fingerprint stored in block 100 no longer matches, nor anything after it. The network rejects the edited version.
2. So that cheating costs the validator money (its stake can be slashed).
3. Contracts and balances are untouched. You only lose from prices moving while you cannot act.

Chapter 3

1. The key is the secret that signs. The address is the public identifier derived from it.
2. An attacker must steal 4 keys instead of 1, and no single person can act alone.

Chapter 4

1. It only reads data, and reading is free. It never sends transactions.
2. Empty code ("0x"): the address is a plain wallet, not a contract.

Chapter 5

1. Ethereum is an L1 with its own validators. Arbitrum is an L2 that runs on top and posts its activity to Ethereum.
2. Solana is not an EVM chain: different programs, RPC methods and address format.

Chapter 6

1. That the published source compiles to the on-chain code. Not that the code is safe or honest.
2. Each answer that is an address becomes an edge, then the crawler asks the new address the same questions (up to 4 hops, 40 contracts).

Chapter 7

1. To fix bugs or add features. Whoever holds the admin role can change the rules, including in their own favour.
2. Time to see a change coming and withdraw before it applies.

Chapter 8

1. 1 USDC is stored as 1,000,000 units.
2. Its backing is volatile crypto or hedges rather than cash. The data shows 38% vs 8.7% depeg episodes per coin-year.
3. It can prove the token supply, transfers and who can mint. It cannot prove the off-chain assets exist.

Chapter 9

1. Locked in the bridge contract on the home chain.
2. An attacker can mint unbacked copies and sell them or post them as collateral. The copy's value collapses and lenders accepting it face bad debt.

Chapter 10

1. The USD value deposited in a protocol's contracts.
2. So that a small project that merely wraps a token is not mistaken for its issuer.

Chapter 11

1. Each unit sold adds to one reserve and the constant product leaves less of the other per unit.
2. 1,200 ETH in the pool, so 2,000,000,000 / 1,200 = 1,666,667 USDC remain. You receive 333,333 USDC (1,667 per ETH): 16.7% impact.

Chapter 12

1. A contract only sees its own chain; it cannot fetch web data.
2. The rate does not fall when the market price falls, so liquidations never start and lenders hold under-collateralized loans.

Chapter 13

1. 10% (the buffer is 1 − LLTV).
2. When the price impact of selling the collateral exceeds the liquidation bonus, or no DEX trades it: they would lose money.
3. 48 / 50 = 96%. Only 2M can be withdrawn now.

Chapter 14

1. Which markets get the money, how much (caps), and in what order.
2. Depositors can withdraw before money is sent to a risky new market.

Chapter 15

1. One unit of the underlying (here sUSDe) per PT.
2. $(1 / 0.98)^{365/182} - 1 \approx 4.1\%$ per year.
3. Liquidators sell the PT into the same Pendle pool, which worsens the price you can exit at.

Chapter 16

1. The gain on one side offsets the loss on the other, so the total stays near the starting dollar value.
2. Part of Ethena's hedges sit on Binance. A Binance failure breaks part of USDe's backing, which hits sUSDe, which is what the PT redeems into.

Chapter 17

1. Margin is the money deposited. Notional is the full position size.
2. Any three of: the Hyperliquid exchange, its L1 chain, its validator oracle, the bridge on Arbitrum holding USDC, the leader's positions.

Chapter 20

1. $1 - e^{-0.1 \times 30/365} \approx 0.82\%$.
2. You keep $0.9 \times 0.8 = 72\%$ of the value, so the loss is 28%.
3. How bad the bad outcomes are: the size of the tail, not just the average.

Chapter 21

1. Few protocols over a short window: having seen no hack does not mean the risk is zero.
2. $(1 + 5 \times 0.087) \div (1 + 5) = 1.435 \div 6 \approx 23.9\%$ per year.

Chapter 22

1. Value edges carry different slices of money, which add up. Common edges are the same single failure seen through several paths.
2. The buffer is 10% and the drop is 25%: $1 - 0.75 \times 0.90 \div 0.90 = 25\%$ shortfall.
3. $5\% \div (1 - 0.5) = 10\%$.

Chapter 23

1. Because it still redeems 1:1 at maturity: below a certain price, buyers lock in a high yield and step in.
2. There is no sale, so no price impact, but only a fraction can be withdrawn and the rest stays trapped through the incident.

Chapter 24

1. $8\% \times 45 \div 365 \approx 0.99\%$.
2. $2\% \div 40\% = 5\%$.
3. Because a lending position crossed its exit trigger u*: freeze risk overrides the average.

Chapter 25

1. Being trapped costs more, so the yield covers the freeze risk only while the hazard is lower, that is, at lower utilization.
2. $1 - \sqrt{(0.002 \times 0.2 \div 0.05)} = 1 - \sqrt{0.008} \approx 91.1\%$.
3. u* = 100%: being trapped costs no more than leaving, so leaving early is never worth it.

Chapter 26

1. Shared draws do not change each pocket's own probabilities (so the average barely moves), but they put losses in the same runs, which fattens the tail.
2. A bridged collateral token (PST on Ethereum and Solana), a stablecoin (USDC), an issuer or its off-chain backing, an exchange, or the same multisig signers.